System Architecture Document

Koopman-Assisted Autonomous Drone Attitude Control

Prahlad Pandav

Contents

1 Introduction

The purpose of this document is to provide a comprehensive and detailed architectural overview of the **Koopman-Assisted Autonomous Drone Attitude Control** project. In this document, we discuss the project's mission objectives, design requirements (in control and physical design), decomposing the drone's functionalities, data and control flow, latency budget, and power architecture.

We address the background of this project as well as the architecturally significant functional requirements set forth by the customer (author). Overall, the intention of this document is to help determine how the quadcopter system will be structured on every level (computational, physical, and algorithms). The System Architecture Document (SAD) will be used throughout the project to ensure that the engineering team is meeting the agreed-upon project requirements throughout the project.

1.1 Background

Autonomy in aerial vehicles is an active area of research in academia and industry. In autonomous aerial vehicles, autonomy is expressed through various applications, including but not limited to: machine vision for obstacle avoidance and path planning (SLAM), control, and decision making. The overarching question that the aforementioned research avenues ask is how aerial vehicles autonomously can navigate complex, harsh, and uncertain environments, especially with real challenges imposed by GPS-denied locations, sensor limitations, and compute resources.

Advancements in aerial/underwater autonomy would open doors to various exciting avenues of scientific endeavors such as space exploration, which this project is geared towards. The recent successful flight of *Ingenuity* on Mars laid the foundations and set the precedent for further research and development of autonomous drone technology, especially for planetary/natural satellite exploration. In such applications of autonomous aerial vehicles, it becomes imperative to implement control over the drone such that mission objectives can be satisfied in spite of challenges posed by lack of GPS navigation or sensor limitations. Therefore, advances in aerial/underwater autonomy would greatly benefit scientific and engineering endeavors for exploring extraterrestrial planets and moon. For instance, Jupiter's moons, Titan and Europa, show great promise for finding evidence of microbiomes due to the presence of water and other chemicals essential for life; for such endeavors, a GPS-denied drone autonomy platform becomes invaluable.

1.2 Purpose

The purpose of this System Architecture Document (SAD) is to define the high-level architecture of the Koopman-Assisted Autonomous Drone Attitude control system and establish the structural framework for hardware and software integration. This document identifies the major system components, their responsibilities, interfaces, communication pathways, and operational assumptions required to achieve autonomous attitude stabilization using a deep learning-assisted optimal control framework.

This SAD serves as the primary reference for system-level design decisions throughout implementation, integration, verification, and future system modifications. By documenting the system architecture prior to hardware integration, the document provides a common understanding of subsystem interactions and reduces integration risk by clearly defining component boundaries, data flow, and design constraints.

This document is intended to guide the implementation of the autonomous flight control system on the target embedded hardware platform while maintaining traceability between the system objectives, architectural decisions, and verification activities.

1.2.1 Problem Definition

The implementation of advanced nonlinear control algorithms on embedded aerial platforms presents challenges that extend beyond flight control law design and simulation validation. Although learning-based reduced-order models and optimal control algorithms can demonstrate satisfactory performance in simulation, successful deployment requires coordinated integration of sensing, state estimation, embedded computation, sensor-computer communication, and flight control hardware operating under real-time constraints.

The objective of this project is to develop an integrated autonomous attitude control system capable of executing a Koopman-assisted Model Predictive Control MPC framework on an embedded computing platform (NVIDIA Jetson Orin Nano Super) while interfacing with a commercial flight controller (Holybro Pixhawk 6C). The architecture must support reliable acquisition of vehicle state-control history, real-time execution of the predictive control architecture, reliable transmission of control commands, and robust operation under practical implementation considerations such as communication latency, timing jitters, computational limitations, sensor uncertainty, actuator constraints, and environmental disturbances.

Unlike simulation environments, the deployed system must operate as a distributed cyber-physical system in which multiple hardware and software subsystems execute concurrently and exchange information through defined communication interfaces. Consequently, a well-defined system architecture is required to partition subsystem responsibilities, establish interface specifications, and support systematic verification and validation prior to experimental flight testing.

1.2.2 Scope

This SAD defines the high-level architecture of the Koopman-assisted autonomous drone attitude control system, including the decomposition of hardware and software subsystems, system responsibilities, communication interfaces, data flow, and architectural constraints required for implementation on the target embedded platform.

The scope of this document includes:

- Overall system architecture and subsystem decomposition.
- Hardware architecture, including the embedded computer, flight controller, sensors, actuators, and communication interfaces.
- Software architecture, including major software modules and their interactions.
- Software architecture, including major software modules and their interactions.
- Control system integration and allocation of subsystem responsibilities.
- Data flow between sensing, state estimation, control, and actuation components.
- System assumptions, operational constraints, and architectural design decisions.
- Considerations for system verification and validation.

This document does not provide detailed mathematical derivations of the control algorithms, machine learning model development, optimization procedures, controller tuning, or software implementation details. These topics are documented separately within the project's research manuscript, software documentation, and verification reports. The architecture described herein represents the baseline system configuration intended to guide implementation, integration, testing, and future system evolution.

1.2.3 Intended Audience

This SAD is intended for engineers, researchers, and collaborators involved in the design, implementation, integration, testing, and maintenance of the autonomous flight control system. The primary audience includes:

- Systems engineers responsible for defining subsystem interactions and system integration.
- Controls engineers implementing and validating the autonomous flight control algorithms.
- Embedded software engineers responsible for deploying the control software on the target computing platform.
- Robotics and autonomous systems researchers evaluating the integration of learning-assisted control methodologies.
- Project collaborators requiring a system-level understanding of the overall architecture.

Readers are expected to possess a general understanding of system dynamics, control theory, embedded computing, autonomous aerial vehicles, and scientific machine learning model development. Detailed knowledge of Koopman Operator Theory is not required.

2 System Overview

2.1 System Objectives

The objective of the Koopman-assisted autonomous drone attitude control system is to provide a modular, real-time flight control architecture capable of stabilizing and controlling the attitude of a quadrotor unmanned aerial vehicle (UAV) using a learning-assisted optimal control framework. The architecture is designed to integrate onboard sensing, state estimation, embedded computation, predictive control, and flight actuation into a cohesive cyber-physical system suitable for autonomous operation. The primary system objectives are as follows:

- Provide autonomous closed-loop stabilization of the quadrotor's roll, pitch, and yaw attitude.
- Track commanded attitude references with high accuracy during nominal flight conditions.
- Execute a learning-assisted predictive control algorithm on an embedded computing platform within real-time computational constraints.
- Utilize a reduced-order representation of the vehicle dynamics to improve computational efficiency while preserving control performance.
- Maintain stable operation in the presence of moderate modeling uncertainty, sensor noise, and external disturbances.
- Interface seamlessly with the flight controller to exchange vehicle state information and control commands.
- Support modular software development by separating sensing, state estimation, predictive modeling, optimization, communication, and data logging into independent functional components.
- Provide an architecture that supports software-in-the-loop (SIL), hardware integration, and experimental flight testing.
- Facilitate future expansion to more advanced autonomous capabilities, including trajectory tracking, adaptive learning, and onboard decision-making.

The architecture emphasizes modularity, maintainability, and scalability such that individual subsystems may be upgraded or replaced with minimal impact on the remainder of the system. This approach enables continued research and development while maintaining a stable and well-defined system foundation for implementation and validation.

2.2 Operational Concept

The autonomous flight control system operates as a distributed embedded control system in which sensing, state estimation, predictive control, and flight actuation execute cooperatively through the interaction of the onboard flight controller and embedded computer. During nominal operation, the

flight controller is responsible for acquiring sensor measurements, estimating the vehicle state, and maintaining low-level flight management functions; in contrast, the companion computer, Jetson Orin Nano Super (hereby referred to as the "Jetson"), executes the learning-assisted predictive controller and generates high-level control commands and desired trajectories.

Upon system startup, power is supplied to the flight controller, the Jetson, sensors, and propulsion system. The flight controller performs hardware initialization, sensor calibration, and state estimator initialization before establishing communication with the Jetson. Once communication has been verified, the Jetson initializes the trained model, predictive controller, and supporting software modules required for autonomous operation. During flight, the operational sequence proceeds continuously as follows:

- Onboard sensors acquire measurements of the vehicle's motion and operating condition.
- The flight controller estimates the current vehicle state using onboard sensor fusion algorithms.
- The estimated vehicle state is transmitted to the Jetson through the communication interface.
- The Jetson evaluates the learned reduced-order dynamics model using the received state information.
- The predictive controller computes the control action based on the current vehicle state, reference command, and predicted future behavior.
- The computed control commands are transmitted back to the flight controller.
- The flight controller converts the received commands into actuator outputs and distributes motor commands to the electronic speed controllers (ESCs).
- Vehicle motion is updated, completing the closed-loop control cycle.
- Flight data, control commands, and diagnostic information are recorded for post-flight analysis and verification.

This closed-loop process repeats continuously throughout autonomous operation at the prescribed control update rate. The operational architecture intentionally partitions computational responsibilities between the flight controller and the Jetson. Time-critical sensor acquisition, state estimation, safety monitoring, and low-level actuator management remain the responsibility of the flight controller, while computationally intensive learning and optimization algorithms execute on the Jetson.

The operational concept also supports a staged development methodology in which the architecture can be validated through software-in-the-loop simulation, hardware integration testing, and incremental flight testing before full autonomous deployment. This approach reduces integration risk by allowing individual subsystems to be verified independently prior to complete system integration. Figure 2.1 presents the control architecture of this project.

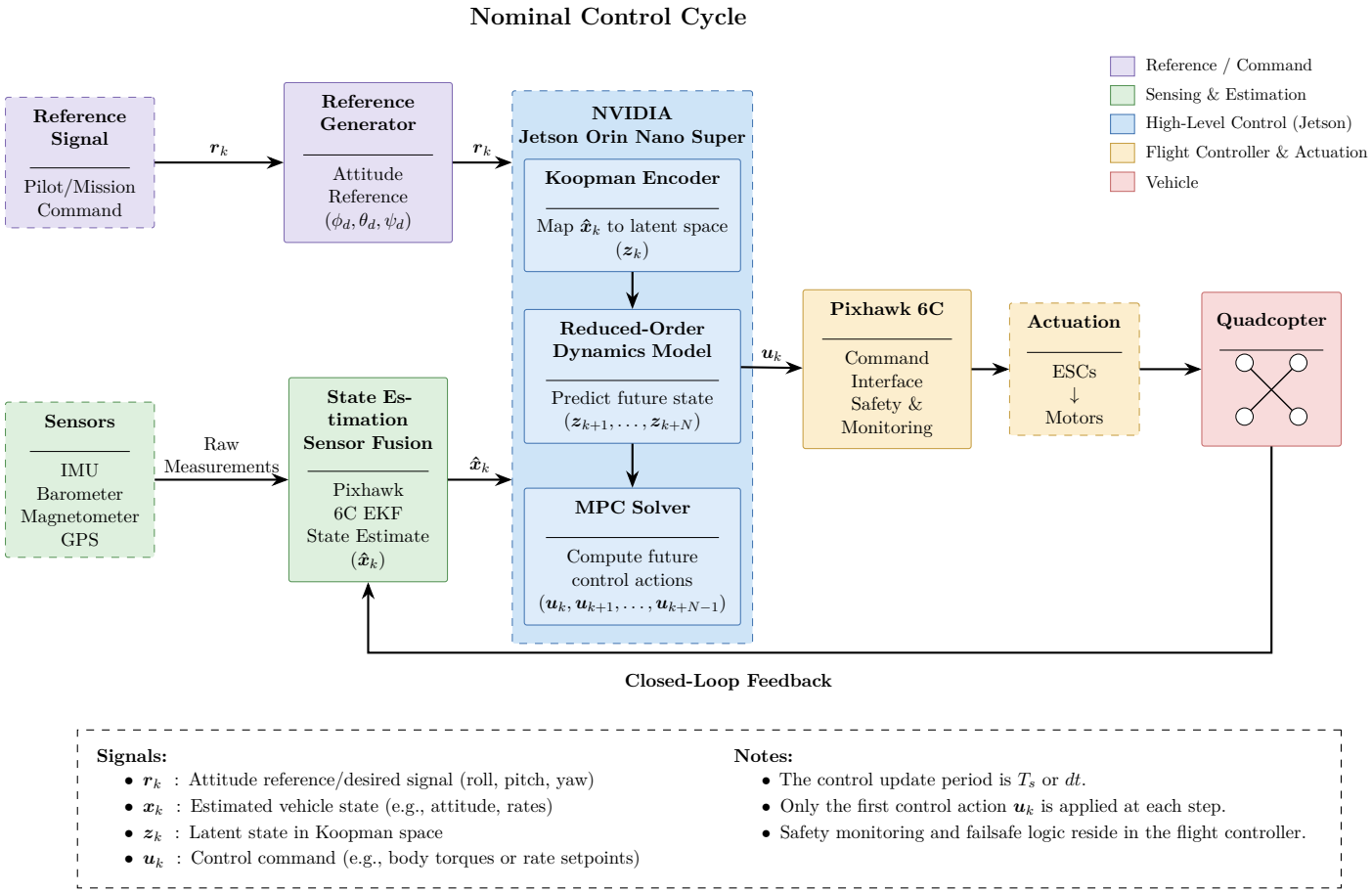


Figure 2.1: Block Diagram Architecture

2.3 System Context

The autonomous drone attitude control system is implemented as a distributed cyber-physical system composed of onboard sensing, embedded computation, flight control hardware, propulsion components, and external operator interfaces. The system is responsible for generating autonomous flight control commands that stabilize the vehicle attitude while operating within the constraints of the Jetson.

Figure 2.2 illustrates the system context and identifies the major internal subsystems and external entities that interact with the autonomous flight control system. The architecture partitions computational responsibilities between the Pixhawk 6C flight controller and the Jetson, allowing learning and optimization architectures to execute independently from low-level flight management functions. The primary system boundary includes the following internal components:

- Embedded computer responsible for executing the learned dynamics model and predictive control algorithm.
- Flight controller responsible for sensor acquisition, state estimation, actuator interfacing, and

safety management.
- Onboard sensors providing measurements of vehicle motion and operating conditions.
- Communication interfaces enabling bidirectional exchange of state information and control commands.
- Electronic speed controllers (ESCs) and propulsion system responsible for vehicle actuation.

Several external entities interact with the autonomous flight control system but are considered outside the architectural boundary. These include:

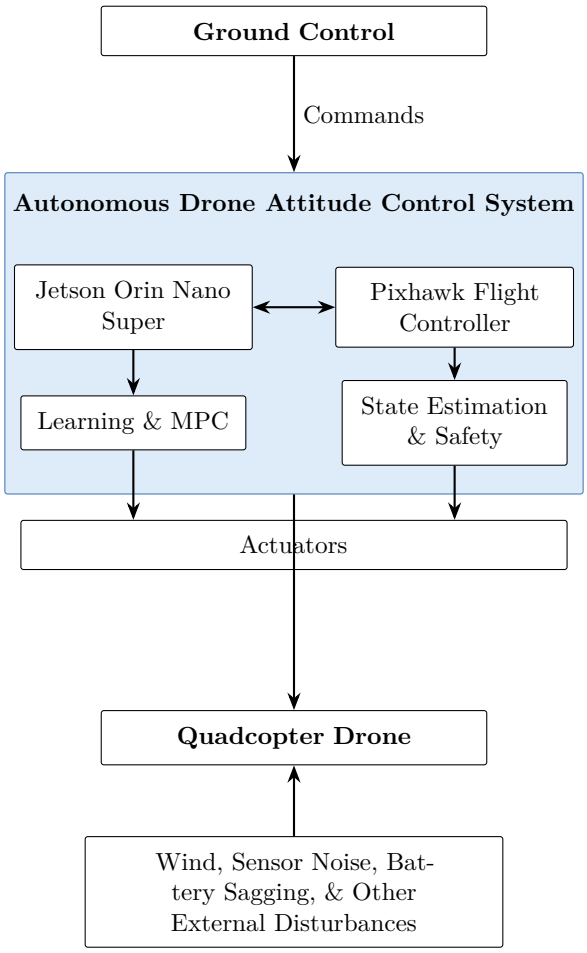
- Operator or Ground Control Station (GCS): Provides mission commands, flight mode selection, system configuration, and telemetry monitoring.
- Mission Reference Source: Supplies desired attitude or trajectory references to the autonomous controller.
- External Environment: Introduces disturbances such as wind gusts, aerodynamic uncertainties, and varying environmental conditions that influence vehicle dynamics.
- Power System: Supplies electrical power to the embedded computer, flight controller, sensors, and propulsion system through the onboard battery and power distribution hardware.

During nominal operation, onboard sensors continuously measure the vehicle's motion and transmit raw measurements to the flight controller. The flight controller estimates the vehicle state using built-in sensor fusion algorithms and communicates the estimated state to the embedded computer. The embedded computer evaluates the learned reduced-order dynamics model and computes an optimal control action using the predictive controller. Control commands are then transmitted back to the flight controller, which performs command validation, control allocation, and actuator interfacing before driving the electronic speed controllers and motors. The resulting vehicle motion closes the feedback loop through the onboard sensors.

The system architecture intentionally separates estimation, low-level flight management, and safety-critical functions from computationally intensive learning and optimization tasks. Partitioning improves modularity, facilitates subsystem verification, and allows future modifications to the control algorithm without requiring significant changes to the flight control hardware or sensing infrastructure.

The system context establishes the architectural boundary used throughout the remainder of this document. Subsequent sections refine this high-level view by describing the hardware architecture, software architecture, communication interfaces, and data flow within the defined system boundary.

Figure 2.2: System Context Diagram



2.4 Major System Components & Functions

The quadcopter drone system is composed of several hardware and software subsystems that collectively provide sensing, state estimation, computation, communication, vehicle actuation, and vehicle motion. Each subsystem performs a defined set of responsibilities while interacting with other subsystems through standardized interfaces. The modular organization of the architecture enables independent development, testing, and future replacement of individual components with minimal impact on the remainder of the system. The following subsections provide a high-level overview of each major subsystem within the autonomous flight control architecture.

2.4.1 Airframe

The airframe serves as the physical platform supporting all onboard hardware, including the propulsion system, flight controller, embedded computer, sensors, power distribution system, and payload components. In addition to providing structural integrity, the airframe establishes the mechanical reference frame used for sensing, state estimation, and control.

A quadrotor configuration is selected for its mechanical simplicity, maneuverability, and ability to independently regulate roll, pitch, yaw, and vertical thrust through differential motor actuation. The airframe functions as the controlled plant whose dynamics are regulated by the autonomous flight control system. The airframe also defines practical design constraints such as mass distribution, structural stiffness, vibration characteristics, payload capacity, and aerodynamic properties that influence overall control performance.

2.4.2 Flight Controller

The flight controller, Pixhawk 6C provides the low-level flight management functions required for safe and reliable operation of the vehicle. It serves as the primary interface between the physical hardware and the high-level autonomous control architecture executing on the embedded computer. The primary responsibilities of the Pixhawk flight controller are:

- Acquisition of onboard sensor measurements.
- State estimation through onboard sensor fusion.
- Management of flight modes and safety functions.
- Reception of high-level control commands from the embedded computer.
- Control allocation and actuator command generation.
- Motor command transmission to the ESCs.

By separating low-level flight management from computationally intensive optimization, the flight controller maintains deterministic operation while providing a stable interface to the higher-level autonomy stack.

2.4.3 Embedded Computer

The embedded computer, Jetson, executes the experimental control architecture required for learning-assisted predictive control. With 1024 CUDA and 32 tensor cores, respectively, the Jetson Orin Nano Super platform provides the processing capability necessary for machine learning inference and optimization-based control. The primary responsibilities of the Jetson are:

- Execution of the learned reduced-order dynamics model.
- Prediction of future vehicle behavior.
- Solution of the predictive control optimization problem.
- Generation of high-level control commands.
- Communication with the flight controller.
- Telemetry logging and diagnostic monitoring.

The embedded computer operates as the decision-making component of the autonomous flight control architecture while relying on the flight controller for sensor acquisition, state estimation, and actuator management.

2.4.4 Sensing

The sensing subsystem within the Pixhawk provides the measurements required for state estimation and closed-loop feedback control. The IMU and GPS sensors observe the vehicle's motion, position, and operating condition throughout flight. These measurements include:

- Angular velocity
- Linear acceleration
- Vehicle orientation
- Atmospheric pressure
- Magnetic heading
- Global position

Sensor measurements are processed by the flight controller to produce an estimate of the vehicle state. These estimated states form the feedback signals used by the predictive controller. The sensing subsystem, therefore, establishes the information pathway connecting the physical vehicle to the autonomous control algorithms.

2.4.5 Communication Interfaces

Communication interfaces provide reliable exchange of measurements and commands between the distributed hardware and software components comprising the autonomous flight control system. The primary communication pathway connects the Pixhawk and the Jetson, allowing the continuous exchange of:

- Estimated vehicle states
- Reference commands
- High-level control inputs
- System status information
- Diagnostic and telemetry data

Additional interfaces support communication with the ground control station for configuration, monitoring, and data retrieval.

2.4.6 Actuation System

The actuation subsystem converts high-level control commands into physical forces and moments acting on the vehicle. The actuation system consists of the electronic speed controllers (ESCs), brushless motors, and propellers responsible for generating thrust.

Upon receiving actuator commands from the Pixhwak, the electronic speed controllers regulate motor speed, producing the differential thrust required to control the vehicle's attitude. The combined action of the propulsion system generates the aerodynamic forces and moments necessary for vehicle stabilization and maneuvering.

The actuation subsystem forms the final stage of the closed-loop control architecture by translating computed control actions into physical vehicle motion. The resulting vehicle response is subsequently measured by the sensing subsystem, thereby completing the feedback loop.

2.4.7 Summary of Components & Responsibilities

Component	Primary Function	Inputs	Outputs
Airframe	Physical Plant	Motor Thrust	Vehicle Motion
Flight Controller	State Estimation and Low-level control	Sensor data, Jetson commands	ESC commands, estimated state
Embedded Computer	High-level autonomy	Estimated state and referene	Control commands
Sensors	Measure vehicle state	Physical motion	Raw measurements
Communication Interface	Data exchange	Messages	Messages
Actuation System	Generate thrust	ESC signals	Forces, moments

Table 2.1: Summary of Components

2.5 Functional Decomposition

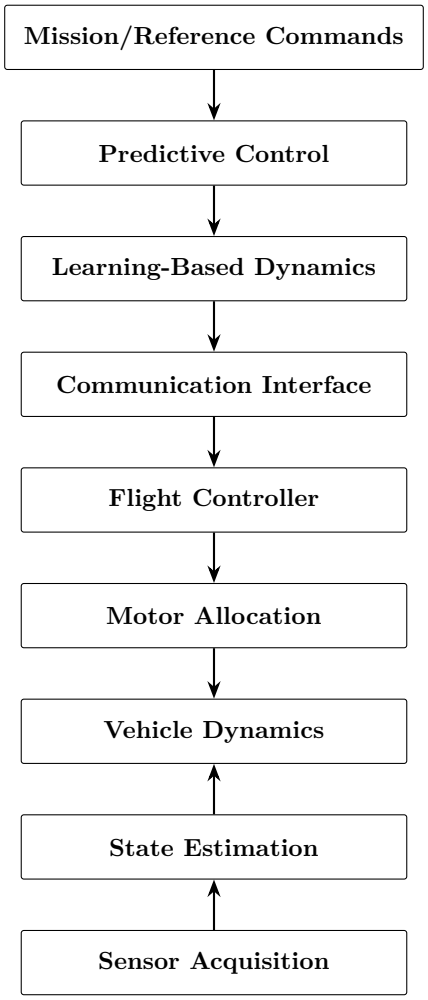
The autonomous drone attitude control system is organized into a hierarchy of functional layers that collectively acquire sensor information, estimate the vehicle state, compute optimal control actions, and actuate the propulsion system. Each layer performs a specific responsibility while exchanging information with adjacent layers through well-defined interfaces. This functional decomposition separates concerns, reduces subsystem coupling, and simplifies system integration and verification. Figure 2.3 illustrates the functional decomposition of the autonomous flight control architecture.

2.5.1 Layer 1: Power Management

The Power Management layer supplies electrical power to all onboard hardware, including the Jetson, communication devices, and propulsion system. It establishes the electrical foundation required for system operation while providing regulated power to each subsystem. The primary functions are:

- Distribute electrical power from the onboard battery
- Supply regulated voltages to electronic subsystems
- Provide power to the propulsion system

Figure 2.3: Functional Decomposition of Autonomous Flight Control Stack



2.5.2 Layer 2: Data Acquisition

The Sensor Acquisition layer measures the physical state of the vehicle and its operating environment through onboard sensing devices. Raw sensor measurements are continuously collected and forwarded to the state estimation subsystem. The primary functions are:

- Measure inertial motion
- Acquire orientation and environmental measurements
- Monitor vehicle operating conditions

2.5.3 Layer 3: State Estimation

The State Estimation layer converts raw sensor measurements into a consistent estimate of the vehicle state suitable for feedback control. Sensor fusion algorithms combine measurements from multiple sensors to improve estimation accuracy and robustness. The primary functions are:

- Fuse measurements from each sensor
- Estimate vehicle attitude and motion states
- Provide feedback variables for the controller

2.5.4 Layer 4: Learning-Based Dynamics

The Learning-Based Dynamics Modeling layer evaluates the learned reduced-order representation of the vehicle dynamics for each control update. The model provides computationally efficient state predictions that support the predictive control algorithm. The primary functions are:

- Evaluate the learned dynamics model
- Generate future state predictions
- Provide reduced-order system linear representation

2.5.5 Layer 5: Predictive Control

The Predictive Control layer computes optimal control actions based on the current vehicle state, desired reference, and predicted system behavior. This layer serves as the primary decision-making component of the autonomous flight control stack. The primary functions are:

- Receive estimated vehicle state
- Receive commanded reference trajectory
- Predict future system evolution
- Solve the optimization problem
- Generate optimal control commands

2.5.6 Layer 6: Communication and Command Interfaces

The Communication and Command Interface layer manages the transmission of information between the Jetson and the Pixhawk. It provides a reliable mechanism for transmitting vehicle states, control commands, diagnostic information, and system status. The primary functions are:

- Exchange state information
- Transmit control commands
- Support telemetry and diagnostics
- Verify communication integrity

2.5.7 Layer 7: Flight Control & Actuation

The Flight Control and Actuation layer converts high-level control commands into actuator signals that generate physical forces and moments on the vehicle. This layer performs control allocation and interfaces directly with the propulsion system. The primary functions are:

- Receive high-level control commands

- Allocate control effort to individual motors
- Generate actuator commands
- Drive the electronic speed controllers

2.5.8 Layer 8: Vehicle Dynamics

The Vehicle Dynamics layer represents the physical response of the quadrotor to actuator inputs. The interaction between the propulsion system and the airframe produces the translational and rotational motion observed by the sensing subsystem, thereby completing the closed-loop feedback process. The primary functions are:

- Respond to actuator inputs
- Generate vehicle motion
- Interact with external disturbances
- Mitigate internal vibrations generated by motors
- Withstand aerodynamic loadings (forces, moments)
- House all hardware components

2.5.9 Functional Interaction

The functional layers operate sequentially during each control cycle. Sensor measurements are acquired and fused to estimate the current vehicle state. The estimated state is transmitted to the Jetson for inference evaluation and predictive control, which compute an optimal control action based on the desired reference and the future time evolution of the quadcopter drone. The resulting control commands are transmitted to the Pixhawk flight controller, converted into actuator commands, and applied to the propulsion system via ESCs. The resulting vehicle motion is measured by the onboard sensors, completing the closed-loop feedback cycle. The mapping of these functional layers to specific hardware and software components is described in the Architecture Description presented in the following section.

2.6 Operational Modes

The autonomous drone attitude control stack operates through a series of defined operational modes that govern subsystem behavior throughout initialization, arming, take-off, hover, flight, landing, and shutdown. Each mode enables a specific set of system functions while ensuring that hardware and software components transition safely between operating states. The operational modes described in this section represent the logical operating states of the autonomous flight control architecture and are independent of any specific flight mode implemented by the flight controller.

2.6.1 Initialization Mode

Initialization Mode begins when power from the batteries is applied. During this mode, the flight controller, embedded computer, sensors, communication interfaces, and supporting software components perform startup procedures required for autonomous operation. Primary activities include:

- Hardware power-up and self-initialization
- Sensor initialization and calibration
- Pixhawk boot sequence
- Jetson startup
- Establishment of communication between the embedded computer and flight controller
- Loading of the learned dynamics model and predictive controller
- Verification of subsystem readiness

The system shall not transition to subsequent operational modes until all required subsystems have completed initialization successfully.

2.6.2 Standby Mode

Standby Mode represents a fully initialized system that is awaiting operator commands or mission execution. During this mode, sensing, communication, and state estimation remain active while propulsion outputs remain disabled or inactive. Primary activities include:

- Continuous sensor monitoring
- State estimation
- Communication health monitoring
- System diagnostics
- Telemetry transmission
- Awaiting operator or mission commands

2.6.3 Autonomous Control Mode

Autonomous Control Mode represents the nominal operating condition of the system during flight. All major subsystems operate concurrently to execute the closed-loop control architecture described in the Operational Concept. While this mode is active, the system continuously performs the following functions:

- Acquire sensor measurements
- Estimate the vehicle state
- Receive reference commands
- Evaluate the learned reduced-order dynamics model
- Compute predictive control actions

- Transmit control commands to the flight controller
- Generate actuator commands
- Record telemetry and diagnostic information

This sequence repeats continuously at the prescribed control update rate throughout autonomous operation.

2.6.4 Fault Management Mode

Fault Management Mode is entered whenever the system detects abnormal or anomalous operating conditions that could compromise safe operation. Examples include communication failures, invalid state estimates, computational faults, or sensor failures. Primary activities include:

- Detection and classification of faults
- Isolation of affected subsystems
- Notification of the operator or ground station
- Execution of predefined fault response procedures
- Preservation of system integrity where possible

Depending on the severity of the detected fault, the system may either recover normal operation or transition to Emergency Mode.

2.6.5 Emergency Mode

Emergency Mode is entered when flight operation is no longer considered safe. The objective of this mode is to avoid vehicle risk and hardware damage. Possible emergency actions include:

- Transfer of control authority from the Jetson back to the Pixhawk flight controller
- Execution of predefined failsafe behaviors
- Controlled descent or landing
- Motor shutdown, when appropriate
- Recording of diagnostic information for post-flight analysis

Emergency Mode remains active until the fault condition is resolved or the system is powered down.

2.6.6 Shutdown Mode

Shutdown Mode represents the controlled termination of system operation following completion of a mission or operator command. Primary activities include:

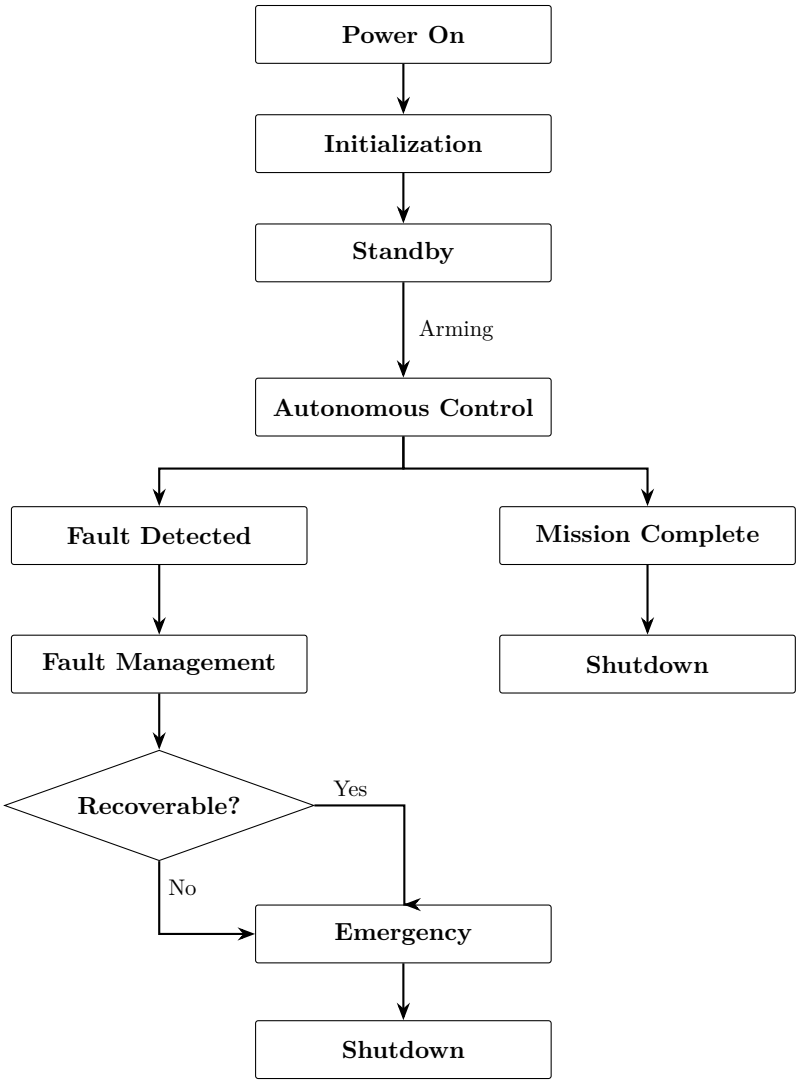
- Termination of autonomous control execution
- Completion of data logging
- Closure of communication interfaces
- Safe shutdown of software services

- Removal of power from onboard subsystems

2.6.7 Operational Mode Transitions

Figure 2.4 illustrates the operational state transitions of the autonomous flight control system. At any point during autonomous operation, the detection of a critical fault may result in a transition to Fault Management Mode or Emergency Mode. Upon successful fault recovery, the system may return to Standby Mode or Autonomous Control Mode, depending on the nature of the fault and operator authorization. The operational modes described above provide a structured framework for coordinating subsystem behavior, simplifying system integration, and supporting verification activities by defining expected system functionality under both nominal and off-nominal operating conditions.

Figure 2.4: Operational Transitions



2.7 High-Level Data Flow

The autonomous drone attitude control system operates as a distributed information processing architecture in which sensor measurements, estimated vehicle states, reference commands, control inputs, and diagnostic information are continuously exchanged between hardware and software subsystems. The high-level data flow defines the logical movement of information throughout the system while abstracting implementation-specific communication protocols and software details. Figure **??** presents the high-level data flow for the autonomous flight control system. The figure illustrates the principal information pathways connecting sensing, state estimation, learning-assisted prediction, optimal control, flight control, and vehicle actuation during nominal operation.

The data flow begins with the acquisition of physical measurements from the onboard sensing subsystem. Inertial, environmental, and navigation sensors continuously observe the dynamic behavior of the vehicle and provide raw measurements to the flight controller. These measurements are processed through onboard state estimation algorithms to produce a time-synchronized estimate of the current vehicle state. The estimated vehicle state is transmitted to the Jetson, where it serves as the primary feedback signal for the autonomous control system. Simultaneously, reference commands supplied by the operator define the desired vehicle attitude or trajectory. Together, the estimated state and reference command provide the inputs required for predictive control.

Within the Jetson, the estimated state is evaluated using the learned reduced-order dynamics model. The reduced-order representation predicts the future time evolution of the vehicle over the optimization horizon while reducing computational complexity that comes with full-order nonlinear dynamic models. These predictions are supplied to the predictive controller, which computes the optimal control action required to minimize tracking error while satisfying system constraints. The resulting control commands are transmitted back to the Pixhawk flight controller through the communication protocols. Upon receiving commands, the flight controller performs command validation, control allocation, and actuator interfacing before generating motor commands for the ESCs. The propulsion system converts these commands into thrust and control moments that produce the physical motion of the quadrotor.

In parallel, the architecture supports several auxiliary data streams that contribute to monitoring and system maintenance. Diagnostic information, communication status, controller outputs, estimated states, and flight telemetry are recorded throughout operation to monitor drone behavior, post-flight analysis, and verification activities. These auxiliary information pathways operate independently of the primary control loop to minimize interference with real-time control performance.

The high-level data flow separates the responsibilities of sensing, estimation, prediction, optimization, communication, and actuation into distinct stages. This separation reduces subsystem coupling, sustains maintainability, and provides clearly defined interfaces that simplify hardware integration and future system modifications. More detailed descriptions of the communication architecture, interface specifications, message formats, and timing considerations are presented in Section 4.

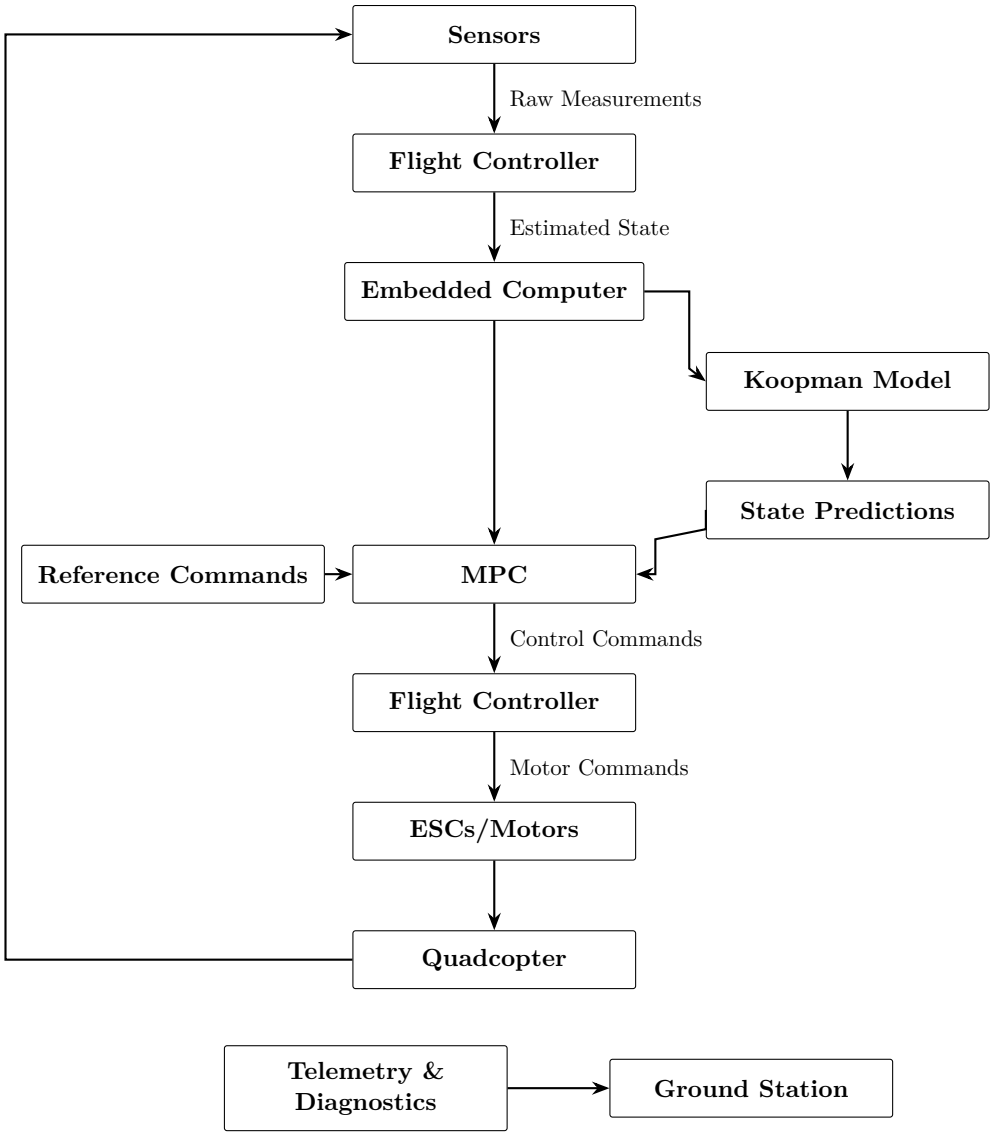


Figure 2.5: High-Level Data Flow

3 Architecture Description

The preceding sections introduced the objectives, operational behavior, functional decomposition, and major components of the autonomous drone attitude control system. This chapter builds upon that foundation by describing how the system is organized to realize those capabilities through a structured hardware and software architecture. The architecture adopts a modular, layered design in which sensing, state estimation, communication, learning-assisted prediction, optimal control, and actuation are partitioned into distinct yet interconnected subsystems. Each subsystem performs a well-defined set of responsibilities and communicates with other components through clearly defined interfaces. This partitioning minimizes subsystem coupling, simplifies system integration, and enables individual components to be developed, tested, and modified with minimal impact on the remainder of the system.

The autonomous flight control stack is implemented as a distributed cyber-physical architecture consisting of a commercial flight controller, an embedded computing platform, onboard sensors, communication interfaces, and the propulsion system. Time-critical flight management functions are executed by the flight controller, while computationally intensive learning and optimization algorithms execute on the embedded computer. The following sections describe the architecture from multiple complementary perspectives. First, the architectural design philosophy establishes the guiding principles that influenced the organization of the system. The subsequent sections describe the overall system architecture, hardware architecture, software architecture, control architecture, and deployment architecture, progressively refining the system from high-level structural organization to the allocation of software components on the target hardware platform.

3.1 Architectural Design Philosophy

The architecture of the autonomous drone attitude control system is founded upon a set of guiding design principles intended to support reliable real-time operation, modular software development, efficient hardware integration, and repeatability. This design philosophy recognizes that autonomous aerial vehicles are inherently multidisciplinary systems in which sensing, estimation, computation, communication, and control operate concurrently. By partitioning these responsibilities, the architecture reduces interdependencies between subsystems and enables each functional component to be developed, verified, and maintained independently. Such modularity also facilitates incremental system integration by allowing individual subsystems to be validated prior to full-system deployment.

There are several fundamental principles guided the architectural design in this project:

- Modularity: Each subsystem performs a distinct set of responsibilities with minimal dependence on the internal implementation of neighboring subsystems.
- Separation of Concerns: Hardware management, state estimation, predictive modeling, op-

timization, communication, and actuation are isolated into dedicated functional layers to simplify development and reduce system complexity.

- Distributed Computation: Computational tasks are allocated according to the capabilities of the underlying hardware. The flight controller performs deterministic, safety-critical functions, while the embedded computer executes computationally intensive machine learning inference and predictive optimization.
- Scalability: The architecture is designed to accommodate future enhancements, including additional sensors, alternative control algorithms, adaptive learning strategies, and higher-level mission planning, without requiring substantial redesign of the overall system.
- Maintainability: Standardized interfaces and well-defined subsystem boundaries allow individual hardware or software components to be upgraded, replaced, or tested independently throughout the system lifecycle.
- Robustness: Functional partitioning and interface abstraction improve fault isolation and support graceful degradation under abnormal operating conditions, thereby contributing to reliable system operation.
- Verification-Oriented Design: The architecture is organized to support staged verification through software-in-the-loop simulation, hardware integration testing, subsystem validation, and experimental flight testing.

While the learning-assisted predictive controller represents the primary contribution of the project, the surrounding system architecture is intentionally designed to remain largely independent of any specific modeling or control methodology. Consequently, future modifications to the dynamics model or control algorithm can be incorporated with minimal impact on the remaining system components, preserving both extensibility and long-term maintainability.

3.2 System Architecture

The autonomous drone attitude control system is organized as a layered, distributed architecture composed of multiple hardware and software subsystems that cooperate to achieve real-time closed-loop attitude stabilization. The architecture distributes responsibilities among dedicated subsystems according to their computational requirements, timing constraints, and safety considerations.

Figure 3.1 presents the high-level system architecture and illustrates the relationships between the principal hardware and software components. The architecture is organized around six primary functional domains:

- Sensing
- State Estimation
- Embedded Computation
- Communication
- Flight Control

- Vehicle Actuation

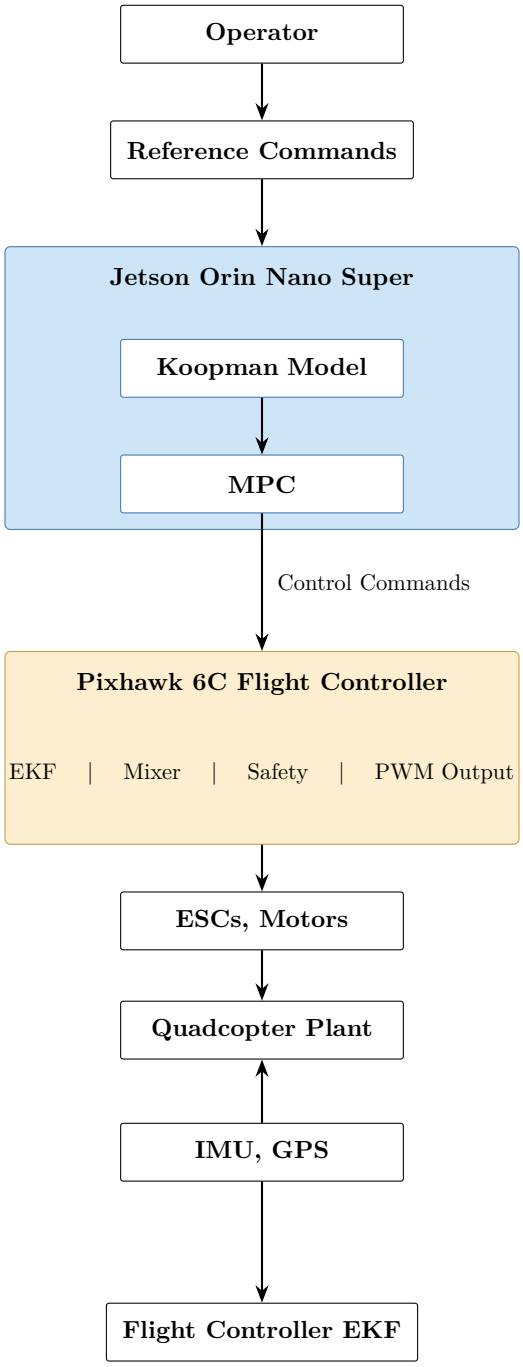


Figure 3.1: System Architecture Diagram

Each domain performs a distinct role within the autonomous control pipeline while interacting with neighboring domains through standardized interfaces.

3.2.1 Sensing Domain

The sensing domain forms the primary source of information for the autonomous flight control system. Onboard sensors continuously measure the physical state of the quadrotor, including inertial motion, orientation, and environmental conditions such as atmospheric pressure. These measurements provide the information needed for state estimation and subsequent control computations. Sensor measurements are transmitted directly to the flight controller, where they undergo preprocessing and sensor fusion prior to transmission to the Jetson.

3.2.2 State Estimation Domain

The state estimation domain is responsible for transforming raw sensor measurements into an estimate of the current vehicle state suitable for feedback control. With its built in sensor fusion algorithms, the Pixhawk flight controller combine and synchronize information from multiple sensing modalities to estimate the vehicle attitude, angular velocity, position, and other state variables required by the predictive controller. The resulting state estimate serves as the primary feedback signal exchanged between the flight controller and embedded computer.

3.2.3 Embedded Computing Domain

The embedded computation domain executes the computationally intensive algorithms associated with learning-assisted predictive control. Upon receiving measurement transmissions from the flight controller, the Jetson embedded computer evaluates the learned reduced-order dynamics model and predicts the future system response over a certain control horizon. These predictions are used by the predictive controller to compute an appropriate control action that drives the drone to a desired state and satisfies the desired control objectives while respecting system constraints. Therefore, the Jetson embedded computer serves as the decision-making component of the entire architecture.

3.2.4 Communication Domain

The communication domain provides the information exchange mechanisms that connect the flight controller, embedded computer, and other electronics into a unified autonomous control system. Estimated vehicle states, reference information, control commands, diagnostic messages, and telemetry data are exchanged through standardized communication interfaces. By abstracting communication from individual subsystem implementations, the architecture supports modular development while simplifying subsystem integration and future hardware replacement. The communication domain also supports system monitoring, diagnostics, and post-flight analysis through continuous telemetry and data logging.

3.2.5 Flight Control Domain

The flight control domain provides execution of low-level flight management functions. After receiving high-level control commands from the Jetson, the flight controller performs command validation, control allocation, actuator interfacing, and safety supervision before generating motor commands for the propulsion system. Maintaining these responsibilities within the flight controller preserves deterministic timing, isolates safety-critical functionality from computationally intensive optimization, and provides a stable interface between autonomous software and vehicle hardware.

3.2.6 Actuation Domain

The vehicle actuation domain converts electrical actuator commands into physical forces and moments acting upon the quadrotor. Electronic speed controllers regulate motor speed in accordance to commands generated by the flight controller. Differential thrust produced by the propulsion system generates the translational and rotational motion required to achieve the desired vehicle attitude. The resulting vehicle motion is measured by the sensing subsystem, thereby completing a given closed-loop update.

Although each functional domain performs an independent responsibility, the overall architecture operates as an integrated closed-loop system. Information flows sequentially from sensing through state estimation, embedded computation, communication, flight control, and vehicle actuation before returning to the sensing subsystem through physical vehicle motion. This organization establishes a clear separation between sensing, computation, communication, and actuation while maintaining continuous feedback loops throughout autonomous operation.

The distributed nature of the architecture enables computationally intensive learning and optimization algorithms to execute independently of deterministic flight management functions. Consequently, modifications to the predictive controller, learned dynamics model, or embedded software can be implemented with minimal impact on the remaining system components, provided that the defined subsystem interfaces remain unchanged. The system architecture, therefore, provides a foundation upon which the hardware, software, and control architectures presented in the following sections are constructed.

3.3 Hardware Architecture

The Hardware Architecture section defines the physical implementation of the autonomous drone attitude control system. It identifies the principal hardware subsystems, their responsibilities, and the interactions required to support real-time autonomous flight. The architecture follows a distributed computing approach in which computational, sensing, communication, and actuation responsibilities are allocated to dedicated hardware components according to their performance requirements and operational constraints. Figure X illustrates the hardware architecture and the

physical interconnections between the major hardware subsystems.

3.3.1 Airframe

The airframe provides the structural foundation of the autonomous flight control system. It supports the integration of the propulsion system, embedded computer, flight controller, sensors, power distribution hardware, and associated communication components. The airframe is designed in a way that supports system identification efforts in order to learn dynamics from data. For instance, the airframe is designed to be stiff so that any vibrations from the motors do not reach the natural frequencies of the airframe. In this way, with the drone subject to aerodynamic loading, the effects of internal shear stresses and bending moments are also mitigated. To reduce weight, use minimal material, and improve cooling to onboard electronics, the fuselage of the drone has a truss-like structure.

In addition to serving as the mechanical structure of the vehicle, the airframe establishes the body-fixed coordinate frame used by the state estimation and control algorithms. Mechanical properties such as mass distribution, structural rigidity, and vibration characteristics directly influence vehicle dynamics and therefore represent important design considerations during system integration. The airframe is also the physical plant within the overall control architecture.

3.3.2 Embedded Computing Platform

The embedded computing platform provides the computational resources required to execute the learning-assisted predictive control framework. The embedded computer is designed to support computationally intensive tasks such as machine learning inference and optimization. Primary responsibilities include:

- Execution of the learned reduced-order dynamics model
- Predictive state propagation
- Solution of the Model Predictive Control (MPC) optimization problem
- Generation of high-level control commands
- Data logging and diagnostic monitoring
- Communication with the flight controller

The embedded computer interfaces directly with the flight controller through the communication subsystem and operates as the primary computational element responsible for autonomous decision making.

3.3.3 Flight Controller

The flight controller provides deterministic real-time management of the vehicle's sensing, estimation, and actuation functions. It serves as the interface between the physical hardware and the high-level

autonomous controller executing on the embedded computer. Primary responsibilities include:

- Acquisition of sensor measurements.
- State estimation through onboard sensor fusion.
- Flight mode management.
- Safety monitoring and failsafe operation.
- Control allocation.
- Generation of motor control signals.
- Communication with the embedded computer.

By retaining safety-critical and timing-sensitive functions within the flight controller, the architecture separates deterministic flight operations from computationally intensive computing.

3.3.4 Sensors

The sensor suite provides continuous measurements of the vehicle's motion and operating environment. These measurements constitute the primary source of information for state estimation and feedback control. The sensing subsystem typically includes:

- Inertial Measurement Unit (IMU)
- Barometric pressure sensor
- Magnetometer
- GPS

3.3.5 Communication Infrastructure

The communication infrastructure provides reliable exchange of information between distributed hardware components within the autonomous flight control system. Primary communication pathways include:

- Flight controller to embedded computer.
- Embedded computer to flight controller.
- Flight controller to ground control station.
- Internal communication between onboard sensors and the flight controller.

Information exchanged through the communication infrastructure includes vehicle state estimates, reference commands, control inputs, telemetry, diagnostic information, and system status messages.

3.3.6 Propulsion & Power System

The propulsion and power system converts electrical energy into the mechanical forces required for vehicle flight while supplying regulated power to all onboard electronic subsystems. The propulsion subsystem consists of electronic speed controllers (ESCs), brushless DC motors, and two pairs of two-blade propellers. The power distribution system consists of a LiPo battery, a power module

dedicated to the flight controller, and voltage regulators. Upon receiving motor commands from the flight controller, the electronic speed controllers regulate motor speed to generate the differential thrust required for vehicle stabilization and maneuvering. The power subsystem simultaneously provides regulated electrical power to the embedded computer, flight controller, and communication devices.

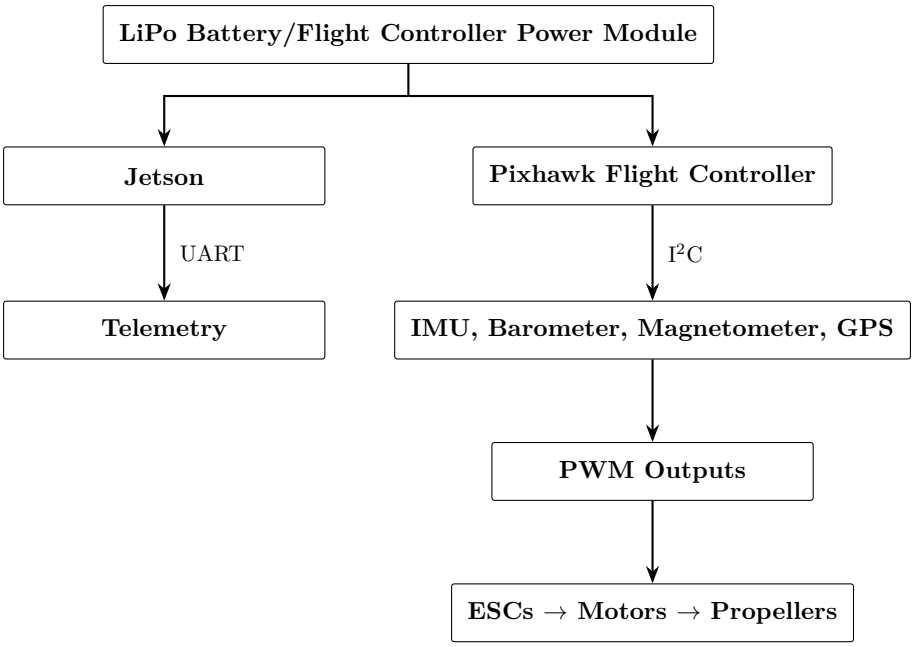


Figure 3.2: Hardware Architecture

3.4 Software Architecture

The software architecture serves as the primary contribution of this project. This section defines the organization of the software components responsible for implementing the autonomous flight control system. The architecture partitions the control software into independent functional modules that communicate through clearly defined interfaces while collectively providing sensing, state management, predictive modeling, optimization, communication, and system monitoring.

The software architecture follows a modular design philosophy in which each software module performs a single primary responsibility. This separation reduces coupling between components, simplifies testing, and enables individual modules to be modified or replaced with minimal impact on the remainder of the software system.

Figure 3.3 illustrates the high-level software architecture and the interactions between the principal software modules.

3.4.1 Software Organization

The autonomous flight control software is organized as a layered processing pipeline that transforms sensor information into control actions through a sequence of estimation, prediction, optimization, communication, and actuation stages. The principal software modules include:

- Mission and Reference Management
- State Interface
- Learning-Based Dynamics Module
- Predictive Control Module
- Communication Module
- Data Logging and Diagnostics

Each module performs a well-defined function while exposing standardized inputs and outputs to neighboring modules. This modular organization improves maintainability and facilitates incremental development and verification.

3.4.2 Mission & Reference Management

The Mission and Reference Management module provides the desired operating objectives for the autonomous controller. Its primary responsibility is to generate or receive reference commands that define the desired vehicle behavior. Depending on the operating mode, these references may originate from an operator, an external mission planner, or a predefined trajectory. Primary responsibilities include:

- Receive reference commands.
- Validate reference inputs.
- Manage mission objectives.
- Provide reference trajectories to the predictive controller.

The module operates independently of the controller implementation, allowing future mission planning capabilities to be incorporated without modifying the lower-level control software.

3.4.3 State Interface

The State Interface module provides a software abstraction between the flight controller and the autonomous controller. Rather than allowing individual software components to communicate directly with the flight controller, the State Interface receives estimated vehicle states, converts data into the internal software representation, and distributes the information to downstream modules. Primary responsibilities include:

- Receive state estimates from the flight controller.
- Validate communication messages.
- Synchronize incoming data.

- Maintain the current system state.
- Provide state information to software modules.

3.4.4 Learning-Based Dynamics

The Learning-Based Dynamics Module implements the learned reduced-order representation of the quadcopter dynamics. Using the current estimated vehicle state and control inputs, the module predicts future system behavior over the optimization horizon. The reduced-order model provides computationally efficient state propagation while preserving the dominant dynamic characteristics required for predictive control. Primary responsibilities include:

- Evaluate the learned dynamics model.
- Generate future state predictions.
- Provide predicted system dynamics to the optimization module.

3.4.5 Predictive Control

The Predictive Control Module serves as the primary decision-making component of the software architecture. Using the estimated vehicle state, reference commands, and predicted future dynamics, the controller computes an optimal control action that minimizes the deviation between desired and actual trajectories while satisfying operational constraints. Primary responsibilities include:

- Receive vehicle state estimates
- Receive reference commands
- Receive predicted system dynamics
- Formulate the optimization problem
- Solve the QP predictive control problem
- Generate high-level control commands

The resulting control commands are then forwarded to the Communication Module for transmission to the flight controller.

3.4.6 Communication Module

The Communication Module manages the exchange of information between the embedded computer and the flight controller. The module abstracts communication protocols from the remaining software architecture by providing a standardized software interface for message transmission and reception. Primary responsibilities include:

- Transmit control commands
- Receive state information
- Monitor communication health
- Handle communication errors

- Synchronize message timing

3.4.7 Telemetry Logging & Diagnostics

The Data Logging and Diagnostics module records system information throughout autonomous operation to support verification, debugging, and performance evaluation. Typical information includes:

- Vehicle state estimates
- Reference commands
- Controller outputs
- Optimization statistics
- Communication status
- Diagnostic messages
- System timing information

The logging subsystem operates independently of the primary control pipeline to minimize interference with real-time execution while providing comprehensive post-flight analysis capabilities.

3.4.8 Software Execution

During nominal operation, the software modules execute continuously as part of the control loop. The execution sequence is as follows:

- Receive estimated vehicle state from the flight controller
- Update the internal system state
- Receive or generate the desired reference command
- Evaluate the learned reduced-order dynamics model
- Predict future vehicle behavior
- Solve the predictive control optimization problem
- Generate high-level control commands
- Transmit commands to the flight controller
- Record diagnostic and performance data

Although these modules are presented sequentially, several execute concurrently during real-time operation. Communication, data logging, and diagnostic monitoring operate in parallel with the primary control pipeline, while the architecture maintains synchronization between software components through defined interfaces and execution timing. The modular organization of the software architecture provides a flexible framework that supports future additions, including alternative learning algorithms, different predictive controllers, additional autonomy functions, or other mission planning capabilities without requiring substantial modification to the surrounding software infrastructure.

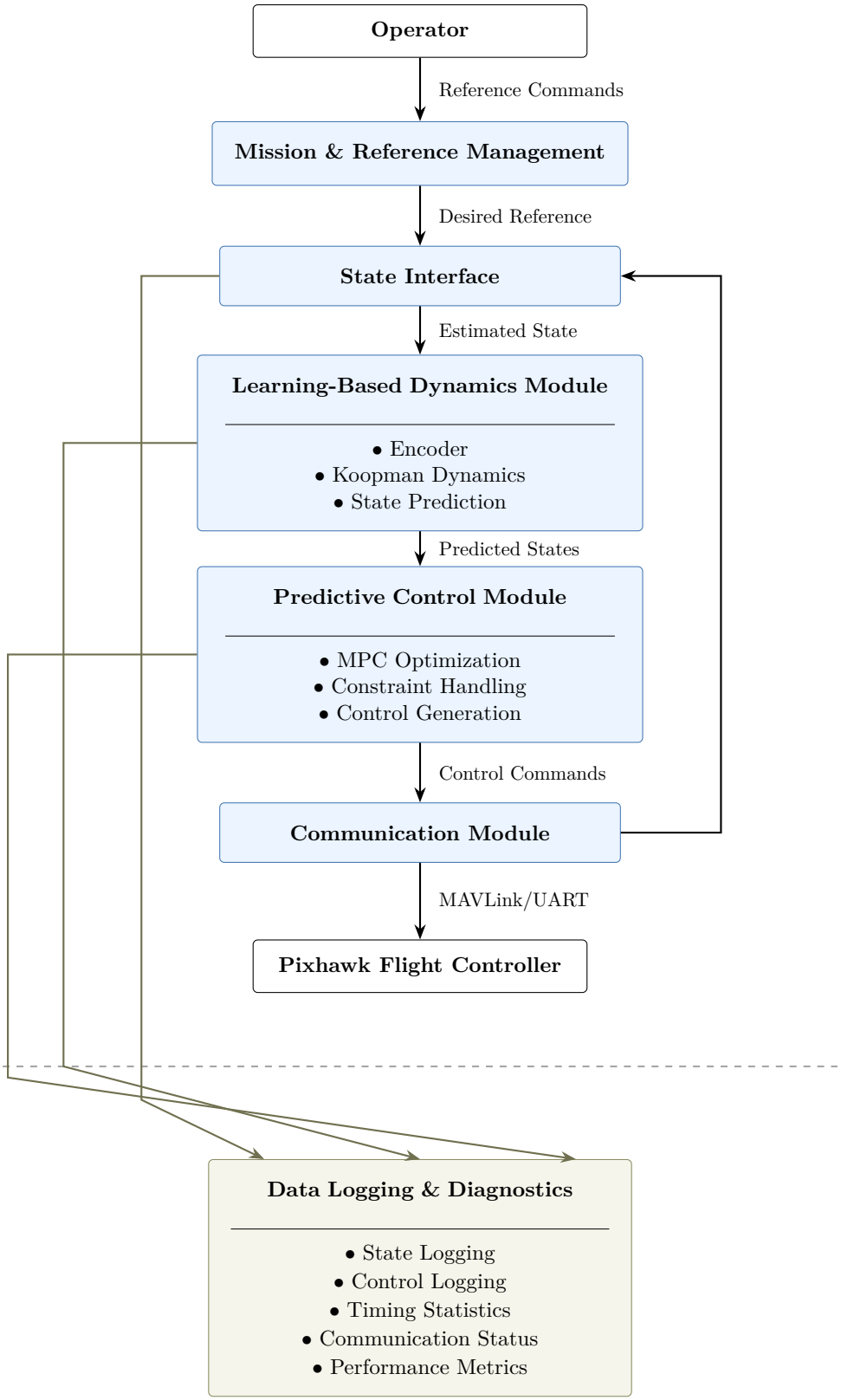


Figure 3.3: Software Architecture Diagram

3.5 Control Architecture

The control architecture defines the organization of the feedback control system responsible for stabilizing the quadrotor and tracking commanded attitude references. The control architecture describes the flow of control information, allocation of control authority, and interaction between the embedded computer and flight controller during closed-loop operation.

The architecture follows a hierarchical control strategy in which predictive control schemes execute on the embedded computer while low-level flight management remains the responsibility of the flight controller. This partitioning combines the computational capability of the embedded platform with the reliability and deterministic execution of the flight controller, resulting in a modular and extensible control system suitable for autonomous flight.

3.5.1 Control System Overview

The autonomous flight control system operates as a closed-loop feedback controller that continuously regulates the vehicle attitude using onboard state measurements and reference commands. During each control cycle, the flight controller estimates the current vehicle state using onboard sensor fusion algorithms. The estimated state is transmitted to the embedded computer, where the learning-assisted predictive controller computes a high-level control command based on the desired reference and predicted future vehicle behavior. The computed control command is transmitted back to the flight controller, which performs control allocation and actuator interfacing before generating motor commands for the propulsion system. Vehicle motion resulting from these commands is measured by the sensing subsystem, completing the feedback loop.

3.5.2 Hierarchical Control Structure

The control architecture is organized into two hierarchical layers. The outer layer (loop) consists of the learning-assisted predictive controller executing on the embedded computer. This layer is responsible for generating optimal high-level control commands using the learned reduced-order dynamics model and the current estimated vehicle state.

The inner layer (loop) consists of the flight controller, which executes deterministic low-level control functions including actuator command generation, motor mixing, and safety supervision. The separation of responsibilities between these layers provides several architectural advantages:

- Isolation of computationally intensive optimization from safety-critical flight functions.
- Improved maintainability through modular subsystem design.
- Independent verification of high-level and low-level control functionality.
- Flexibility for future controller development without modifying the underlying flight control hardware.

The hierarchical organization also simplifies hardware integration by defining a clear interface

between autonomous control software and flight management functions.

3.5.3 Outer-Loop Predictive Control

The outer-loop controller executes on the embedded computer and represents the primary decision-making element of the autonomous flight control system. Using the estimated vehicle state and desired reference command, the controller evaluates the learned reduced-order dynamics model to predict future vehicle behavior over the optimization horizon. The predictive controller then computes a high-level control action that minimizes the deviations from desired performance while satisfying operational constraints. Primary responsibilities include:

- Receive estimated vehicle state
- Receive desired reference commands
- Evaluate the learned dynamics model
- Predict future vehicle behavior
- Solve the predictive optimization problem
- Generate high-level control commands

The outer-loop controller is designed to remain independent of the flight controller.

3.5.4 Inner-Loop Flight Controller

The inner-loop controller executes on the flight controller and provides deterministic low-level flight management functions. The flight controller receives high-level input commands from the embedded computer and converts them into actuator signals suitable for the propulsion system. Primary responsibilities include:

- Receive high-level control commands
- Validate incoming commands
- Perform control allocation
- Generate motor control signals
- Monitor actuator limits
- Execute safety and failsafe functions

By retaining these responsibilities within the flight controller, the architecture preserves deterministic execution and isolates safety-critical functions from higher-level autonomy software.

3.5.5 Feedback Architecture

The control architecture employs a discrete-time closed-loop feedback structure that utilizes estimated vehicle states to regulate system behavior. The feedback process consists of the following sequence:

- Vehicle motion is measured by onboard sensors.
- The flight controller estimates, synchronizes, and fuses current vehicle state.

- The estimated state is transmitted to the embedded computer.
- The predictive controller computes the optimal control action.
- High-level control commands are transmitted to the flight controller.
- The flight controller generates actuator commands.
- The propulsion system produces the corresponding vehicle motion.

This feedback cycle repeats continuously throughout operation, allowing the controller to compensate for disturbances, modeling uncertainty, and changing operating conditions. The architecture maintains a single feedback pathway between the flight controller and embedded computer, simplifying interface management.

3.5.6 Control Authority & Responsibility Allocation

The control architecture partitions control authority between the embedded computer and the flight controller in accordance to subsystem responsibilities and system requirements. The embedded computer is responsible for learning-based dynamics prediction, predictive optimization, generation of high-level control commands, and autonomous decision making. On the contrary, the flight controller is responsible for measurements and sensor fusion, state estimation, flight mode management, control allocation, motor command generation, safety monitoring, and failsafe operation.

This partitioning establishes a clear architectural boundary between high-level autonomous decision making and low-level deterministic flight control. The resulting control architecture provides a scalable framework for integrating advanced learning-assisted control algorithms while maintaining the reliability, determinism, and safety characteristics required for autonomous aerial vehicles.

3.6 Deployment Architecture

The deployment architecture defines how the software components and control functions of the autonomous drone attitude control system are allocated to the hardware platform. Whereas the Hardware Architecture identifies the physical subsystems and the Software Architecture describes the organization of computational modules, the Deployment Architecture establishes the mapping between these architectural views by specifying where each software module executes and how the distributed computing platform supports real-time autonomous flight.

The autonomous flight control stack employs a distributed embedded architecture consisting of two primary computational platforms: an embedded computer responsible for learning-assisted prediction and optimal control, and a dedicated flight controller responsible for flight management, state estimation, and actuator interfacing. These platforms communicate through a standardized communication interface, allowing computational tasks to be partitioned according to their timing requirements, computational complexity, and safety considerations. This deployment strategy enables computationally intensive algorithms to execute independently of time-critical flight management functions while maintaining a deterministic closed-loop control system. By separating

high-level autonomous decision making from low-level flight control, the architecture improves maintainability, facilitates subsystem verification, and simplifies future hardware upgrades without requiring substantial modification to the overall system organization. The following sections describe the deployment philosophy, the allocation of software modules to each computational platform, the communication infrastructure connecting the distributed hardware, and the runtime execution strategy that coordinates autonomous operation.

3.6.1 Deployment Overview

The deployment architecture follows a distributed processing paradigm in which software components are allocated according to the capabilities and responsibilities of the underlying hardware. The embedded computer serves as the primary autonomy processor. It executes the learning-assisted dynamics model, predictive controller, mission management logic, communication services, and diagnostic software. These components require comparatively high computational throughput and memory resources while operating at the control update rate established for autonomous flight. The flight controller provides deterministic execution of safety-critical functions that directly interface with the physical vehicle. Data acquisition, state estimation, sensor fusion, flight mode management, actuator command generation, and failsafe monitoring remain resident on the flight controller throughout all phases of operation.

Communication between the two computational platforms is performed through a bidirectional interface that continuously transmits estimated vehicle states, reference information, high-level control commands, diagnostic messages, and system status information. This communication pathway enables the embedded computer to make autonomous control decisions while allowing the flight controller to retain authority over hardware interfacing and low-level vehicle management. Figure X illustrates the deployment architecture of the autonomous flight control system, including the allocation of software modules to the embedded computer and flight controller, together with the communication interfaces connecting the distributed computing platform.

By establishing clear ownership of computational responsibilities, the deployment architecture provides a robust foundation for incremental integration, subsystem verification, and future system expansion. This allocation strategy also supports staged testing, allowing individual hardware and software components to be validated independently before complete system integration and experimental flight testing.

3.6.2 Embedded Computer Deployment

The embedded computer serves as the primary computational platform for autonomous decision making within the flight control architecture. Software components requiring substantial computational resources are deployed on the embedded computer, allowing advanced prediction and optimization algorithms to execute without compromising the deterministic operation of the flight

controller.

The embedded computer hosts the high-level autonomy software responsible for learning-assisted predictive control. These software modules operate cooperatively to transform estimated vehicle states and reference commands into high-level control actions suitable for autonomous flight. Software modules deployed on the embedded computer include:

- Mission and Reference Management
- State Interface
- Learning-Based Dynamics Module
- Predictive Control Module
- Communication Module
- Data Logging and Diagnostics

During nominal operation, the embedded computer receives the estimated vehicle state from the flight controller through the communication interface. The State Interface validates and synchronizes the received information before distributing it to the remaining software modules. Reference commands supplied by the Mission and Reference Management module are combined with the current vehicle state to define the desired operating objective.

The Learning-Based Dynamics Module evaluates the reduced-order vehicle model and predicts future system behavior over the prediction horizon. These predictions are supplied to the Predictive Control Module, which computes the optimal control action according to the specified objective function and operational constraints. Once the control solution has been obtained, the Communication Module transmits the resulting high-level control commands to the flight controller. Simultaneously, the Data Logging and Diagnostics module records controller outputs, state estimates, communication statistics, and performance metrics for post-flight analysis and system verification.

The embedded computer is isolated from direct hardware control. It neither acquires raw sensor measurements nor directly interfaces with the propulsion system; it relies upon the flight controller to provide validated state information and execute low-level actuator commands. This architectural separation helps with maintaining and changing computationally intensive autonomy algorithms independently of hardware-specific flight management functions.

3.6.3 Flight Controller Deployment

The flight controller provides the deterministic execution environment required for low-level flight management and hardware interfacing. The flight controller is dedicated to real-time sensing, state estimation, actuator management, and safety supervision. The flight controller hosts the software components responsible for direct interaction with the physical vehicle. These components execute continuously throughout system operation and maintain deterministic timing independent of the computational workload executing on the embedded computer. Software functions deployed on the flight controller include:

- Sensor Drivers
- State Estimation
- Flight Mode Management
- Control Allocation
- Motor Mixing
- PWM Generation
- Safety Monitoring and Failsafe Logic

Raw measurements obtained from the onboard sensor suite are processed by the sensor drivers and supplied to the onboard state estimation algorithms. The resulting estimate of the vehicle state is made available to both the internal flight control functions and the embedded computer through the communication interface. Upon receiving high-level control commands from the embedded computer, the flight controller validates the incoming transmission before performing control allocation and motor mixing. The resulting actuator commands are converted into pulse-width modulation (PWM) signals that regulate the ESCs and the propulsion system.

The flight controller also maintains responsibility for flight mode management and continuous safety monitoring. Communication integrity, estimator validity, actuator limits, and other system health indicators are continuously evaluated to ensure safe operation. If abnormal operating conditions are detected, the flight controller executes predefined fault response procedures independently of the embedded computer. For instance, if, during flight-safety checks, it is found that communication lines between the flight controller and embedded computer are faulty, the flight controller retains control authority and returns the drone to the landing zone.

3.6.4 Communication Deployment

The autonomous flight control system employs a distributed computing architecture in which the embedded computer and flight controller execute independent software services while continuously exchanging information through a bidirectional communication interface. The communication deployment defines the mechanisms by which these computational platforms coordinate sensing, prediction, optimization, and actuation activities during autonomous operation. Communication between the embedded computer and flight controller consists primarily of three information pathways:

- Vehicle state information transmitted from the flight controller to the embedded computer.
- High-level control commands transmitted from the embedded computer to the flight controller.
- Diagnostic and status information exchanged between both platforms.

The flight controller periodically transmits estimated vehicle states generated by the onboard state estimation subsystem. These messages provide the embedded computer with the information required to evaluate the learned dynamics model and compute predictive control actions. Upon receiving the estimated state, the embedded computer performs predictive modeling and optimization

before generating high-level control commands. These commands are subsequently transmitted back to the flight controller, where they are validated and converted into actuator signals suitable for the propulsion system. In addition to the primary control pathway, auxiliary communication channels support telemetry, system monitoring, synchronization, and diagnostic functions. Communication health indicators, controller statistics, computational timing information, and subsystem status messages are exchanged throughout operation to support system supervision and post-flight analysis.

Communication between distributed software components is therefore treated as a service layer within the deployment architecture rather than as a direct dependency between individual modules. This organization simplifies subsystem integration, improves maintainability, and facilitates future hardware modifications.

Figure 3.4 illustrates the communication deployment architecture and the transmission pathways between the embedded computer and flight controller.

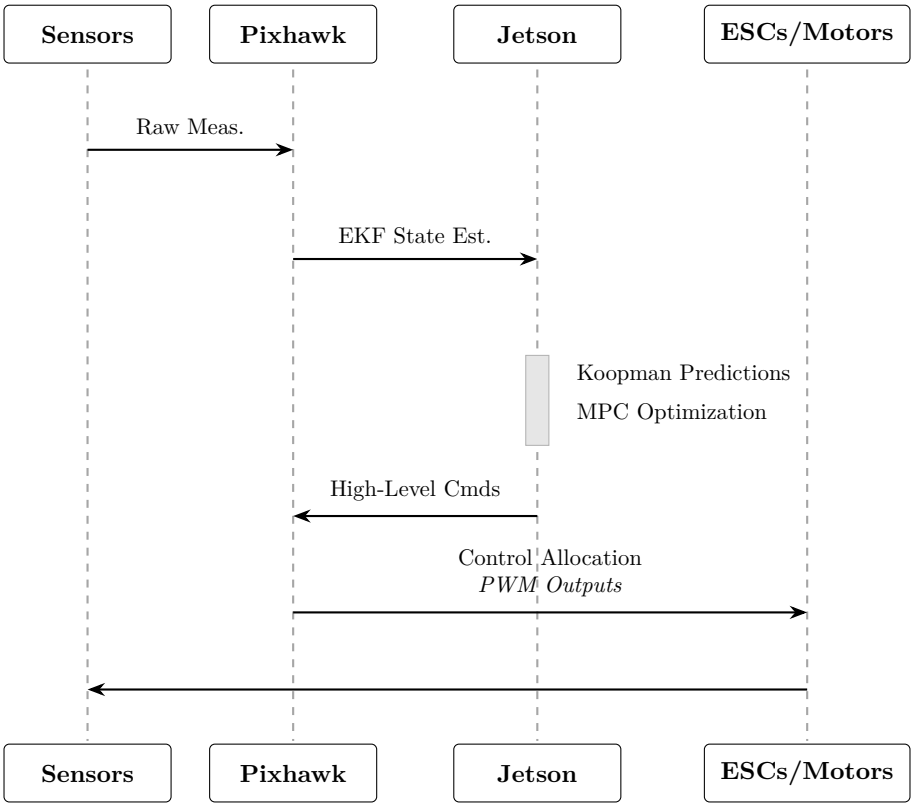


Figure 3.4: Communication Sequence Diagram

3.6.5 Runtime Execution

The runtime execution architecture defines the temporal behavior of the autonomous flight control system during operation. Execution begins during initialization, when both computational platforms perform startup procedures and establish communication. The flight controller initializes onboard sensors, sensor fusion and state estimation algorithms, and safety monitoring services, while the

embedded computer loads the learned dynamics model, predictive controller, communication services, and supporting software modules. Following successful initialization, the system transitions into nominal autonomous operation. During each control cycle, the following sequence is executed:

- Sensor measurements are acquired by the flight controller.
- The flight controller estimates the current vehicle state.
- Estimated state information is transmitted to the embedded computer.
- The embedded computer updates the internal state representation.
- The learned dynamics model predicts future vehicle behavior.
- The predictive controller computes the optimal control action.
- High-level control commands are transmitted to the flight controller.
- The flight controller validates incoming commands and generates actuator signals.
- The propulsion system produces the resulting vehicle motion.
- System telemetry and diagnostic information are recorded.

This execution sequence repeats continuously throughout autonomous operation, forming the closed-loop feedback cycle responsible for vehicle stabilization. Although the control pipeline is conceptually sequential, several software services execute concurrently during runtime. Communication services, data logging, diagnostics, and health monitoring operate in parallel with the primary control loop. Similarly, state estimation and actuator management continue to execute independently of the computational workload associated with predictive control.

3.6.6 Resource Allocation & Scalability

The autonomous drone attitude control system has been designed to efficiently allocate computational responsibilities among the available hardware resources while maintaining sufficient flexibility to support future system expansion. The distributed deployment architecture intentionally separates computationally intensive autonomy functions from deterministic flight-critical operations, thereby reducing subsystem coupling and maintaining clear boundaries between each subsystem's responsibilities.

Resource allocation is primarily driven by the computational characteristics and timing requirements of individual software components. Functions requiring deterministic execution and direct hardware interaction, including data acquisition, state estimation, actuator management, and safety supervision, are allocated to the flight controller. These functions operate under strict real-time constraints and therefore benefit from execution on a dedicated embedded platform specifically designed for flight-critical applications.

Conversely, computationally intensive functions such as learned dynamics evaluation, predictive state propagation, optimization-based control, diagnostic analysis, and data logging are allocated to the embedded computer. This allocation leverages the substantially greater processing capability and memory resources available on the embedded computing platform while preserving deterministic

operation of the flight controller. The resulting deployment architecture provides several advantages:

- Isolation of high computational workloads from safety-critical tasks.
- Reduced risk of timing interference between autonomous software and low-level flight functions.
- Simplified subsystem verification and debugging.
- Independent hardware and software development.
- Improved maintainability through clear functional partitioning.

The architecture further supports incremental resource expansion through its modular organization. Since software modules communicate through well-defined interfaces, additional functionality may be incorporated with minimal modification to existing subsystems. Examples of potential future enhancements include:

- Additional onboard sensors and sensing modalities.
- Higher-level trajectory planning and mission management capabilities.
- Adaptive or online learning algorithms.
- Alternative predictive control methodologies.
- Obstacle avoidance and autonomous navigation functions.
- Multi-vehicle coordination and cooperative control strategies.
- Additional diagnostic and health-monitoring services.

The distributed deployment strategy also facilitates future hardware upgrades. Modifications to the embedded computing platform, flight controller, communication interfaces, or sensing hardware can be accommodated without substantial redesign of the surrounding software architecture, provided that subsystem interfaces remain consistent.

From a computational perspective, the architecture is designed to support real-time execution through functional partitioning and parallel execution of independent services. Furthermore, the architecture supports staged development and validation by enabling individual software modules and hardware subsystems to be developed, tested, and deployed independently. This capability reduces integration risk and allows computational resources to be evaluated progressively as system complexity increases.

Overall, the resource allocation strategy provides a scalable and extensible architectural foundation capable of supporting both the current autonomous attitude control objectives and future research developments. The modular organization, distributed computation model, and clearly defined subsystem interfaces ensure that future enhancements can be incorporated while preserving the fundamental architectural structure established throughout this document.

4 Interfaces & Data Flow

The autonomous drone attitude control system employs a distributed architecture consisting of multiple hardware and software subsystems that cooperate through well-defined interfaces. These interfaces enable the exchange of sensing measurements, estimated vehicle states, control commands, diagnostic information, and system status data required to support autonomous operation (e.g., battery charge level, embedded computer CPU/GPU temperatures, ESC temperatures, etc.). The objective of the interface architecture is to ensure reliable communication between subsystems while minimizing coupling between hardware-specific implementations and higher-level autonomy functions. The principal interfaces within the autonomous flight control system can be categorized into four primary groups: hardware, software, communication, and data flow interfaces. Collectively, these interfaces establish the information pathways necessary to support sensing, state estimation, predictive control, actuation, diagnostics, and system supervision. Figure 4.1 presents the Interface Context Diagram of the autonomous drone control system architecture.

4.1 Interface Overview

The purpose of this chapter is to define the interfaces that enable communication between sensing, estimation, computation, control, and actuation components within the autonomous flight control architecture. Clearly defined interfaces establish subsystem boundaries, reduce coupling between components, and facilitate independent development, integration, testing, and future system modifications.

The interface architecture follows a modular design philosophy in which each subsystem interacts with neighboring components through standardized communication pathways rather than direct implementation dependencies. This approach promotes maintainability, improves fault isolation, and simplifies future upgrades to hardware and software components.

4.2 Hardware Interfaces

Hardware interfaces define the physical connections between onboard components and specify the mechanisms through which electrical signals and communication protocols are exchanged. These interfaces establish the physical foundation of the distributed computing architecture. Examples include:

- Sensor interfaces with the flight controller.
- Communication links between the embedded computer and flight controller.
- Flight controller interfaces with the electronic speed controllers.
- Power distribution interfaces supporting onboard electronics.

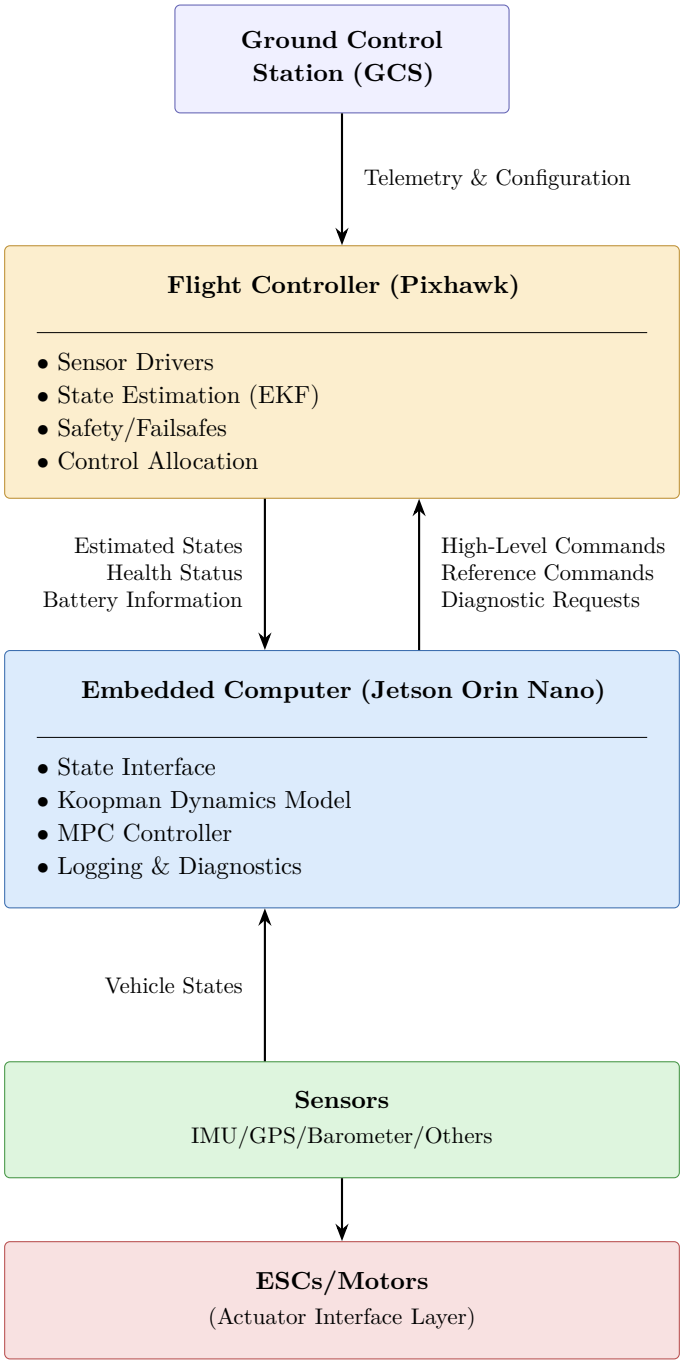


Figure 4.1: Interface Context Diagram

4.3 Software Interfaces

Software interfaces define the logical interactions between software modules executing on the embedded computer and flight controller. These interfaces specify the information required by individual software services while abstracting implementation-specific details from neighboring modules. Examples include:

- State information provided to the predictive controller.
- Reference commands supplied to the autonomy stack.
- Predicted states exchanged between modeling and optimization modules.
- Diagnostic information provided to monitoring and logging services.

4.4 Communication Interfaces

Communication interfaces define the exchange of information between distributed computational platforms and supporting systems. The primary communication pathways within the autonomous flight control system include:

- Flight Controller $\rightarrow$ Embedded Computer
- Embedded Computer $\rightarrow$ Flight Controller
- Ground Control Station $\leftrightarrow$ Flight Controller
- Ground Control Station $\leftrightarrow$ Embedded Computer

Information exchanged through these interfaces includes:

- Estimated vehicle states
- High-level control commands
- Reference information
- System status messages
- Diagnostic information
- Telemetry

The communication architecture enables the embedded computer to execute computationally intensive autonomy algorithms while preserving deterministic flight management functions on the flight controller.

4.5 Data Flow Architecture

Data interfaces define the ownership, structure, and flow of information throughout the autonomous control architecture. During nominal operation, information propagates through the system according to the following sequence:

- Sensors acquire measurements of vehicle motion and operating conditions.
- The flight controller estimates the current vehicle state.

- Estimated state information is transmitted to the embedded computer.
- The embedded computer evaluates the learned dynamics model and computes the optimal control action.
- High-level control commands are transmitted back to the flight controller.
- The flight controller generates actuator commands for the propulsion system.
- Telemetry and diagnostic information are recorded throughout operation.

This information flow establishes the primary closed-loop pathway responsible for autonomous flight.

4.6 Timing & Synchronization

The autonomous drone attitude control system is implemented as a distributed sampled-data control architecture in which sensing, state estimation, predictive control, and actuation are executed on separate platforms operating at varying update frequencies. Consequently, proper timing management and synchronization of measurements are essential to ensure reliable closed-loop operation and maintain control performance. Unlike purely simulated environments, practical implementation introduces computational delays, communication latencies, asynchronous execution, and timing uncertainties that may degrade system performance if not properly considered during system integration.

4.6.1 Sampling Architecture

The autonomous flight control system consists of multiple periodic processes operating at different execution frequencies. These processes include sensor acquisition, state estimation, predictive control computation, communication, and actuator command generation. Representative update frequencies are summarized in the following table:

Process	**Nominal Frequency**
IMU Sampling	$250 - 1000\ Hz$
State Estimation & EKF	$100 - 400\ Hz$
Embedded Computer State Updates	$50\ Hz$
Koopman Prediction & MPC	$50\ Hz$
Flight Controller Inner Loop	$250 - 400\ Hz$
PWM/DShot Actuation	$400 - 600\ Hz$
Telemetry Logging	$50\ Hz$

Table 4.1: Range of Update Frequencies for each Subsystem

4.6.2 Distributed Execution Timing

The architecture partitions computational responsibilities between the flight controller and embedded computer. The flight controller executes deterministic low-level flight management functions, including sensor acquisition, state estimation, safety monitoring, and actuator command generation. The embedded computer executes computationally intensive learning and optimization algorithms. This distributed architecture introduces additional timing considerations, which collectively contribute to overall control loop delay, including:

- Communication latency between computational platforms.
- Variable computation time associated with optimization routines.
- Message scheduling and transmission delays.
- Synchronization uncertainty between asynchronous processes.

4.6.3 Synchronization Considerations

To maintain consistency between subsystems, exchanged information shall be associated with appropriate timestamps whenever possible. Time synchronization enables detection of stale measurements, estimation of communication delays, synchronizing logged measurements at each time step, and consistent reconstruction of drone flight behavior during post-flight analysis. Subsystem synchronization considerations include:

- State estimate freshness
- Controller execution deadlines
- Command update rates
- Telemetry timestamp consistency

During operation, the embedded computer assumes that state estimates received from the flight controller are sufficiently recent for predictive control computations.

4.6.4 Communication Latency

Communication between the flight controller and embedded computer introduces non-negligible delays that must be considered during controller implementation. Sources of latency include sensor processing delays, state estimation delays, message serialization and transmission, embedded computation time, and operating system scheduling delays. If communication latency proves to be excessive, the effects may result in degraded tracking performance of the controller, reduction in stability margins, increased control effort with, and stale command execution. Consequently, system integration must seek to minimize end-to-end latency and ensure that computational deadlines are satisfied during nominal operation.

4.6.5 Timing Assumptions

The architecture is developed under the following timing assumptions. Violation of these assumptions may adversely affect autonomous flight performance and shall therefore be considered during verification and flight testing activities:

- State estimates are available at a sufficiently high frequency for predictive control.
- Communication delays remain bounded during nominal operation.
- Predictive controller execution completes within its designated sampling interval.
- Timing jitter remains sufficiently small relative to controller sampling periods.
- Flight controller inner-loop execution remains deterministic and unaffected by embedded computer workloads.

4.7 Interface Requirements

The interfaces defined within the autonomous drone attitude control architecture shall satisfy the functional and performance requirements necessary to support reliable autonomous operation, subsystem interoperability, and future system expansion. The following interface requirements govern the exchange of information between subsystems.

4.7.1 State Information

IR-01: The flight controller shall provide estimated vehicle state information to the embedded computer during autonomous operation.

IR-02: State information shall include, at minimum, vehicle attitude, angular rates, and associated timestamps.

IR-03: State information shall be updated at a frequency sufficient to support real-time predictive control execution.

4.7.2 Control Command

IR-04: The embedded computer shall transmit high-level control commands to the flight controller.

IR-05: The flight controller shall remain responsible for low-level actuator command generation and control allocation.

IR-06: Communication interfaces shall support bidirectional information exchange between computational platforms.

4.7.3 Communication

IR-07: Communication interfaces shall support detection of communication loss and stale information.

IR-08: Communication latency shall remain bounded during nominal operation.

IR-09: Interfaces shall support timestamped data exchange whenever feasible.

IR-10: Communication interfaces shall support diagnostic and health monitoring information.

4.7.4 Software

IR-11: Software modules shall exchange information through clearly defined interfaces.

IR-12: Interfaces shall minimize coupling between subsystem implementations.

IR-13: Interface modifications shall minimize impacts on neighboring software modules.

4.7.5 Fault Tolerance

IR-14: Interface failures shall not directly result in uncontrolled actuator behavior.

IR-15: Communication failures shall permit transition to predefined fault management procedures.

IR-16: The flight controller shall retain authority to execute safety and failsafe functions independent of embedded computer operation.

4.7.6 Logging & Diagnostics

IR-17: Interfaces shall support telemetry and diagnostic data collection.

IR-18: Logged information shall permit post-flight reconstruction of system behavior and timing characteristics.

IR-19: Interface definitions shall support future integration of additional sensors, autonomy modules, and computational services.

5 Assumptions and Constraints

The autonomous drone attitude control system has been developed under a set of assumptions and design constraints that influence architectural decisions and design, subsystem allocation, and expected system performance. These assumptions and constraints establish the conditions under which the system is intended to operate and define the boundaries within which the architecture is expected to satisfy its functional and performance requirements.

As discussed in Chapter 4, unlike simulation environments, practical implementation introduces numerous limitations arising from computational resources, communication delays, sensor uncertainties, environmental disturbances, and hardware constraints. Consequently, the architecture presented in this document should be interpreted within the context of these assumptions and limitations. The purpose of this chapter is to document the assumptions made during system development, identify hardware, software, and operational limitations that influenced architectural decisions, establish expected operating conditions for autonomous flight, and identify potential sources of uncertainty and failure that may impact system performance during implementation and testing.

Explicit identification of assumptions and constraints facilitates system verification, supports future design modifications, and provides traceability between architectural decisions and the conditions under which those decisions were made. The following sections discuss the principal assumptions and constraints associated with the autonomous drone control system architecture.

5.1 Design Assumptions

The autonomous drone attitude control architecture has been developed under several modeling, computational, and control-related assumptions. These assumptions simplify system development and establish the conditions under which the proposed architecture is expected to achieve acceptable performance.

5.1.1 Vehicle Dynamics

The quadcopter is assumed to behave as a rigid-body system whose dominant dynamics can be adequately represented using conventional translational and rotational governing equations. Structural flexure and aeroelastic effects are assumed to have negligible influence on the primary attitude control dynamics. Propulsion system dynamics are assumed to be significantly faster than the outer-loop predictive control dynamics, allowing motor and propeller responses to be approximated as sufficiently rapid relative to controller sampling rates.

5.1.2 Modeling Assumptions

The learned Koopman-based dynamics model is assumed to adequately capture the dominant nonlinear behavior of the vehicle within the operating conditions represented by the training data. The following modeling assumptions are made:

- Training data sufficiently represents expected operating conditions.
- The reduced-order latent representation preserves the dominant system dynamics relevant to attitude control.
- Prediction errors remain bounded during nominal operation.
- Significant extrapolation beyond the training domain does not occur during autonomous flight.

The architecture therefore assumes that the learned dynamics model remains valid within the intended flight envelope.

5.1.3 State Estimation

The architecture assumes that state estimates provided by the flight controller are sufficiently accurate and available at update rates compatible with predictive control execution. Specifically, it is assumed that:

- Sensor measurements are properly calibrated.
- State estimation algorithms remain stable during nominal operation.
- Estimation errors remain sufficiently small relative to control objectives.
- State information is communicated with acceptable communication latency.

The predictive controller is therefore designed under the assumption that estimated states adequately represent the true vehicle state with reasonable variance.

5.1.4 Compute

The distributed architecture assumes that computational resources available on the embedded computer are sufficient to execute the learning-based predictive controller within its designated sampling interval. The following assumptions are made:

- Neural network inference latency remains bounded.
- Optimization routines converge within acceptable execution times.
- Processor utilization remains below levels that would significantly degrade real-time performance.
- Memory resources remain sufficient for autonomous operation.

5.1.5 Control Architecture

The hierarchical control architecture assumes continued availability of low-level stabilization functions provided by the flight controller. Specifically, it is assumed that:

- Inner-loop attitude stabilization remains operational.
- The flight controller maintains actuator authority.
- High-level commands generated by the embedded computer remain physically achievable.
- Communication delays remain sufficiently small to preserve stability margins.

Violation of these assumptions may adversely affect system performance and shall therefore be considered during verification and flight testing activities.

5.2 Operational Assumptions

The autonomous drone attitude control system is intended to operate under a set of expected environmental and mission conditions. The following operational assumptions define the conditions under which the proposed architecture is expected to function as intended.

5.2.1 Nominal Flight Envelope

Autonomous operation is assumed to occur within the flight envelope represented during system modeling, simulation, and controller development. The vehicle is therefore assumed to operate within:

- Expected attitude limits
- Expected angular rate limits
- Expected actuator capabilities
- Expected environmental disturbance levels

Operation significantly outside these conditions may reduce inference performance, prediction accuracy, and controller performance.

5.2.2 Mission

The system is intended primarily for experimental validation and research-oriented autonomous flight operations. Accordingly, it is assumed that:

- Mission objectives remain physically feasible.
- Reference trajectories remain dynamically achievable.
- Autonomous operation occurs in controlled testing environments.
- Flight durations remain consistent with available energy resources.

5.2.3 Operator

Since the conception of this project, the presence of an operator during system testing and evaluation has been assumed. The following assumptions are therefore made:

- A human operator remains available during flight testing.
- Manual intervention may be initiated if abnormal behavior is observed.
- Vehicle initialization, calibration, and arming procedures are performed prior to autonomous operation.
- Appropriate pre-flight checks are conducted.

5.2.4 Communication

The distributed architecture assumes continued communication between computational platforms during autonomous operation. Specifically, it is assumed that:

- Communication links remain available during nominal operation.
- Packet losses remain infrequent.
- Communication delays remain bounded.
- Timing jitter remains sufficiently small relative to controller sampling periods.

5.2.5 Environmental

The quadcopter is assumed to operate under environmental conditions that do not exceed the capabilities of onboard hardware and software. Examples include:

- Moderate wind disturbances
- Acceptable temperature ranges
- Nominal sensor operating conditions
- Limited electromagnetic interference

Extreme environmental conditions may adversely affect estimation accuracy, communication reliability, and overall control performance.

5.3 Safety Assumptions

The autonomous drone attitude control system is developed as an experimental research platform and therefore relies upon several safety-related assumptions regarding system operation, human supervision, and subsystem behavior. These assumptions establish the conditions under which autonomous flight testing is expected to be conducted and define the boundaries of acceptable operational risk.

5.3.1 Human Supervision

The current development phase assumes that all autonomous flight operations are conducted under continuous human supervision. Specifically, it is assumed that:

- An operator remains present during testing activities.
- The operator maintains situational awareness throughout autonomous operation.
- Manual intervention may be initiated if abnormal system behavior is observed.
- Flight testing is conducted in controlled environments appropriate for experimental operations.

The architecture, therefore, does not presently assume fully unsupervised or beyond-visual-line-of-sight operation.

5.3.2 Flight Controller Safety

The architecture assumes that the flight controller maintains independent low-level flight management and safety capabilities. These capabilities include:

- Sensor monitoring
- State estimation
- Flight mode management
- Failsafe logic
- Emergency disarming functionality
- Actuator command generation

The architecture is designed such that the flight controller retains authority over actuator commands and safety-critical functions regardless of the operational status of the companion embedded computer, especially if an abnormality is detected.

5.3.3 Communication Failure

The distributed control architecture assumes that communication failures between the embedded computer and flight controller can be detected and appropriately managed. The following assumptions are made:

- Communication loss can be identified within an acceptable time interval.
- Predefined recovery actions are available in the event of communication failure.
- Loss of communication does not directly result in uncontrolled actuator behavior.

5.3.4 Computational Safety

The embedded computer is assumed to execute learning and optimization algorithms without interfering with safety-critical flight controller functions. The following assumptions are made:

- Excessive processor utilization on the embedded computer does not compromise flight controller

operation.
- Software failures occurring on the embedded computer remain isolated from low-level stabilization functions.
- Controller execution deadlines are sufficiently bounded to avoid unsafe command generation.

5.3.5 Flight Testing

The development phase assumes progressive validation through incremental testing activities, including, but not limited to,

- Monte Carlo testing
- Software-in-the-loop testing
- Bench testing
- Hardware integration testing
- Tethered flight testing
- Controlled field autonomous flight testing

Higher-risk operational scenarios are assumed to be deferred until lower-level verification activities have demonstrated acceptable performance. These safety assumptions establish the operational context under which autonomous testing is intended to occur and provide the basis for future verification and risk assessment activities.

5.4 Hardware Constraints

The architecture of the autonomous drone attitude control system is influenced by numerous hardware limitations associated with computational resources, mass properties, power availability, sensing capabilities, and communication infrastructure. These constraints significantly impact subsystem allocation decisions and the practical implementation of learning-based predictive control.

5.4.1 Mass & Packaging

The quadcopter is subject to finite payload and geometric limitations that constrain the selection and placement of onboard hardware components. These constraints include:

- Available payload capacity
- Structural limitations of the airframe
- Center of mass requirements
- Component packaging and mounting considerations
- Wiring and connector routing limitations

5.4.2 Compute

The embedded computing platform possesses finite processing and memory resources that directly influence the complexity of onboard autonomy algorithms. Computational limitations include:

- Available CPU and GPU resources
- Memory capacity limitations
- Thermal constraints
- Real-time execution requirements
- Operating system scheduling limitations

These constraints motivate the use of reduced-order modeling techniques and computationally efficient learning architectures capable of satisfying real-time execution requirements. Similarly, the flight controller possesses limited computational resources and is therefore primarily allocated deterministic low-level functions, including state estimation, safety management, and actuator command generation.

5.4.3 Power

Autonomous operation is fundamentally limited by the available onboard energy resources. Power-related constraints include:

- Finite battery capacity;
- Battery discharge limitations;
- Voltage sag during high-power maneuvers;
- Thermal effects associated with high current draw;
- Available power for computational hardware and sensors.

The architecture, therefore, assumes that computational workloads and mission durations remain compatible with the available onboard energy budget.

5.4.4 Sensing

Sensor performance imposes additional limitations on autonomous operation. Examples include:

- Measurement and process noise
- Finite sensor update rates
- Varying sensor biases, uncertainties, and drift
- Resolution limitations
- Calibration uncertainties
- Sensitivity to vibration and environmental disturbances

These limitations directly influence state estimation performance and consequently affect predictive controller accuracy. Therefore, it is imperative that the human operator correctly calibrates every

sensor before flight.

5.4.5 Communication & Hardware

Communication interfaces are similarly constrained by available hardware resources. Examples include:

- Available serial interfaces
- Communication bandwidth limitations
- Message throughput limitations
- Synchronization uncertainties
- Physical connector and wiring limitations

These constraints influence achievable update frequencies and motivate the partitioning of computational responsibilities between the flight controller and embedded computer.

5.4.6 Thermal

Both computational hardware and power electronics are subject to thermal limitations that may impact sustained autonomous operation. Examples include:

- Embedded computer thermal throttling
- Power distribution heating
- Electronic speed controller temperature limits
- Battery temperature limitations

Consequently, computational workloads and mission profiles must remain compatible with the thermal capabilities of onboard hardware. As a design consideration, the quadcopter is designed as a truss-like structure with structural loading members, which promotes forced convection cooling of compute and power distribution hardware during flight.

5.5 Software Constraints

The autonomous drone attitude control system relies upon multiple software components executing across heterogeneous computational platforms. Consequently, the overall architecture is subject to various software limitations associated with real-time execution, operating system behavior, numerical methods, and third-party software dependencies. These constraints significantly influence controller implementation, subsystem allocation, and achievable autonomous performance.

5.5.1 Real-Time Execution

The autonomous control architecture operates as a sampled-data system in which sensing, state estimation, communication, and control computations must be completed within designated sampling

intervals. Software execution is therefore constrained by:

- Controller execution deadlines;
- Operating system scheduling delays;
- Variability in computational workloads;
- Communication synchronization requirements;
- Timing jitter associated with asynchronous processes.

Failure to satisfy execution deadlines may result in stale information, increased control effort, degraded tracking performance, or potential loss of stability margins; in such an event, if a timing deadline is missed, the flight controller retains control authority over all subsystems and safely guides the quadcopter back to the landing zone. Consequently, computationally intensive tasks are allocated to the embedded computer, while deterministic low-level flight management functions remain on the flight controller.

5.5.2 Operating System

The embedded computer utilizes a general-purpose operating system (Linux) that does not inherently provide hard real-time guarantees. This poses several limitations, including:

- Process scheduling uncertainty
- Non-deterministic task execution times
- Background operating system processes
- Resource contention between software services
- Potential timing jitter

As a result, the architecture assumes that software execution delays remain sufficiently bounded such that autonomous control performance remains acceptable.

5.5.3 Computational Resources

Software complexity is constrained by the finite computational resources available on the embedded computer and flight controller. Examples include:

- Available CPU resources
- GPU utilization limitations
- Memory capacity limitations
- Data storage limitations
- Numerical solver execution times

These constraints motivate the use of computationally efficient reduced-order models and learning architectures capable of satisfying real-time execution requirements.

5.5.4 Numerical Constraints

The predictive control architecture depends upon numerous numerical algorithms whose performance may be influenced by finite precision effects and solver limitations. Examples include:

- Floating-point precision limitations
- Numerical conditioning of optimization problems
- Solver convergence limitations
- Accumulation of prediction errors
- Approximation errors associated with reduced-order modeling

Consequently, controller performance may degrade if numerical assumptions are violated.

5.5.5 Software Dependencies

The architecture additionally relies upon external software frameworks and third-party libraries. Examples include:

- Flight controller firmware dependencies
- Machine learning frameworks
- Communication middleware
- Numerical optimization libraries
- Operating system services

Changes to these dependencies, if needed, may require modifications to software interfaces or computational workflows.

5.5.6 Development & Maintainability

Software development is further constrained by maintainability considerations, including:

- Modularity requirements
- Interface compatibility
- Version management
- Testing requirements
- Expansion for future autonomy capabilities.

Accordingly, the architecture emphasizes modular subsystem design and clearly defined software interfaces to reduce implementation complexity and facilitate future development.

5.6 Communication Constraints

The autonomous drone attitude control system employs a distributed computing architecture in which state estimation, predictive control, and actuator management are executed on separate

computational platforms. Consequently, communication performance plays a significant role in determining overall autonomous system behavior. Communication limitations directly influence control latency, synchronization accuracy, and achievable controller bandwidth.

5.6.1 Communication Latency

Information exchanged between the flight controller and embedded computer experiences finite transmission delays arising from:

- Message serialization
- Interface transmission delays
- Operating system scheduling
- Message buffering
- Software processing overhead

These delays contribute to the overall closed-loop control latency and may reduce available stability margins if not appropriately considered during controller design. The architecture therefore assumes that communication delays remain sufficiently bounded relative to controller sampling periods.

5.6.2 Bandwidth Limitations

Communication interfaces possess finite throughput capabilities that limit the amount of information that may be exchanged between subsystems. Examples include limitations associated with:

- Serial communication rates
- Message transmission frequencies
- Telemetry bandwidth
- Logging throughput
- Simultaneous communication services

Bandwidth limitations may restrict achievable update rates and influence the quantity of information exchanged between computational platforms.

5.6.3 Timing Jitter

Communication processes are inherently asynchronous and may exhibit variability in message arrival times. Sources of timing jitter include:

- Operating system scheduling uncertainty
- Variable computational workloads
- Communication buffering effects
- Resource contention between software services

Excessive timing jitter may result in irregular control updates, stale state information, less confidence

in prediction accuracy and uncertainty, and degraded control performance. The architecture, therefore, assumes that timing variability remains sufficiently small relative to controller execution frequencies.

5.6.4 Packet Loss & Communication Reliability

Communication interfaces may occasionally experience dropped messages or temporary interruptions. Potential causes include:

- Communication congestion
- Software failures
- Electrical interference
- Hardware faults
- Resource limitations

The architecture, therefore, assumes that packet losses remain infrequent during nominal operation, communication failures are detectable, and recovery actions are available in the event of communication degradation.

5.6.5 Synchronization

Because the autonomous architecture consists of multiple asynchronous processes, precise synchronization between computational platforms cannot be guaranteed. Synchronization limitations include:

- Clock drift between computational platforms
- Timestamp uncertainty
- Delayed state information
- Misalignment of logged data

Consequently, subsystem timing information should be timestamped whenever possible to facilitate synchronization and post-flight analysis.

5.7 Environmental Constraints

The autonomous drone attitude control system is intended to operate within environmental conditions that are compatible with the capabilities of the onboard hardware, sensing systems, and control algorithms. Environmental factors may significantly influence vehicle dynamics, state estimation accuracy, communication reliability, and overall autonomous performance. Consequently, environmental effects represent an important consideration during system design, verification, and flight testing activities.

5.7.1 Atmospheric Disturbances

The vehicle is subject to external disturbances arising from atmospheric effects such as steady wind conditions, Wind gusts, turbulent air, and local airflow disturbances generated by nearby structures. These disturbances introduce uncertainties into the vehicle dynamics and may degrade tracking performance or increase control effort requirements. The predictive controller is, therefore, expected to operate under moderate disturbance conditions representative of the intended operating environment.

5.7.2 Temperature Effects

Onboard electronic systems are subject to performance variations resulting from ambient temperature conditions. Potential temperature-related effects include:

- Sensor bias drift
- Reduced battery performance
- Thermal throttling of computational hardware
- Increased electrical resistance and power losses

Operation outside the recommended temperature ranges of onboard components may adversely affect autonomous system performance and reliability, and cause onboard harware to fail themselves.

5.7.3 Sensor Environmental Effects

Environmental conditions may directly influence sensor accuracy and measurement quality. Examples include:

- Vibrational disturbances affecting inertial measurements
- Electromagnetic interference
- Temperature-induced sensor drift
- Reduced measurement quality due to adverse weather conditions

These effects may degrade state estimation performance and subsequently impact predictive controller accuracy.

5.7.4 Power System Environmental Effects

Battery performance is influenced by environmental conditions and operational loading. Examples include:

- Temperature-dependent battery characteristics
- Voltage sag during aggressive maneuvers
- Reduced available capacity under adverse environmental conditions

Power system limitations may therefore indirectly constrain achievable mission duration and control

performance.

5.7.5 Operating Environment

The current development phase assumes operation within relatively controlled environments suitable for experimental validation. Consequently, the architecture does not presently assume operation under:

- Severe weather conditions
- Extreme wind environments
- Significant precipitation
- Highly cluttered electromagnetic environments

Future development efforts will require additional robustness measures to support operation in more challenging environments.

5.7.6 Environmental Uncertainty

The autonomous architecture is ultimately subject to environmental uncertainties that cannot be completely eliminated or accurately modeled. Examples include:

- Unmodeled aerodynamic effects
- Time-varying disturbances and parameter
- Sensor degradation
- Changes in vehicle characteristics over time

Accordingly, verification activities should evaluate system robustness under representative environmental conditions to determine the practical limits of autonomous operation.

5.8 Architectural Limitations & Tradeoffs

The autonomous drone attitude control system has been developed through a series of engineering decisions intended to balance computational performance, safety, modularity, and implementation complexity. Consequently, the resulting architecture reflects numerous tradeoffs that influence system capabilities and limitations. The following subsections summarize the principal architectural decisions and their associated benefits and limitations.

5.8.1 Distributed Computing

The proposed system partitions computational responsibilities between the flight controller and embedded computer. This architecture provides several advantages:

- Separation of safety-critical and computationally intensive functions
- Improved modularity and maintainability

- Scalability for future autonomy capabilities
- Reduced computational burden on the flight controller

However, this decision also introduces several limitations such as increased communication complexity, additional synchronization requirements, communication-induced latency, and additional integration and verification effort. Consequently, autonomous performance is partially dependent upon communication reliability and timing behavior.

5.8.2 Retention of Flight Controller Authority

The architecture intentionally preserves flight controller authority over low-level stabilization and actuator command generation. Advantages of this approach include:

- Preservation of established flight safety mechanisms
- Availability of independent failsafe functionality
- Simplified emergency recovery procedures
- Reduced implementation risk during early development phases

However, this decision also introduces several limitations:

- Reduced direct control authority for the predictive controller
- Additional communication interfaces
- Potential restrictions on achievable control bandwidth
- Increased dependence on interaction between multiple control layers

5.8.3 Learning-Based Dynamics Modeling

The use of a learned Koopman-based dynamics model enables computationally efficient prediction of nonlinear vehicle behavior. Advantages include:

- Improved representation of nonlinear dynamics
- Reduced computational complexity relative to high-fidelity nonlinear models
- Potential adaptability to complex system behaviors

However, several limitations remain:

- High dependence on training data quality and coverage
- Reduced interpretability compared to physics-based models
- Potential degradation under distribution shift
- Limited extrapolation capability beyond training conditions
- Discretization inherently removes some dynamical information of the drone system.

The predictive controller therefore assumes that operational conditions remain reasonably consistent with the data used during model development.

5.8.4 Reduced-Order Modeling

The architecture employs reduced-order representations to satisfy real-time computational requirements. Advantages include:

- Reduced computational requirements
- Improved real-time feasibility
- Lower memory requirements

Limitations include:

- Approximation errors
- Potential loss of higher-order dynamics
- Reduced fidelity under aggressive operating conditions

Selection of model complexity, therefore, represents a compromise between computational efficiency and predictive accuracy.

5.8.5 Hierarchical Control Architecture

The use of a hierarchical control structure permits separation of high-level predictive control from low-level stabilization functions. Advantages include:

- Modular development
- Simplified subsystem verification
- Independent development of autonomy functions
- Improved fault isolation

Limitations include:

- Increased architectural complexity;
- Additional timing dependencies;
- Potential interactions between control layers;
- Additional tuning and integration effort.

5.8.6 Simulation-To-Implementation Gap

Although extensive simulation studies may demonstrate acceptable controller performance, practical implementation introduces additional complexities that are difficult to fully represent in simulation environments. Examples include:

- Communication delays
- Timing jitter
- Sensor imperfections
- Computational limitations
- Environmental disturbances

- Hardware integration effects

Accordingly, successful simulation results do not guarantee equivalent real-world performance, and significant verification and validation activities remain necessary prior to operational deployment.

5.8.7 Current Limitations

The current development phase remains subject to several limitations, including:

- Dependence on supervised testing activities
- Limited validation under real-world operating conditions
- Incomplete characterization of communication-induced effects
- Limited robustness evaluation outside nominal operating conditions

These limitations represent opportunities for future development and motivate continued system integration, verification, and flight testing activities. Despite these limitations, the proposed architecture provides a modular and scalable foundation for the implementation and evaluation of learning-assisted autonomous flight control systems.

6 V&V Considerations

The autonomous drone attitude control system described throughout this document represents a complex cyberphysical architecture consisting of tightly coupled hardware, software, communication, and control subsystems. Therefore, demonstration of acceptable system performance requires more than validation of individual algorithms or components in isolation.

The objective of verification and validation activities is to establish confidence that the implemented system satisfies its functional and performance requirements while operating within the assumptions and constraints identified in previous chapters. Verification activities focus on determining whether the system has been implemented correctly and whether subsystem interactions conform to architectural expectations. Validation activities focus on determining whether the resulting system adequately satisfies the intended operational objectives and performs acceptably under representative operating conditions.

Due to the distributed and learning-assisted nature of the proposed architecture, verification and validation must consider additional implementation factors that are often neglected in simulation environments, including:

- Communication delays and synchronization uncertainties
- Computational timing constraints
- Sensor imperfections and estimation errors
- Hardware limitations and environmental disturbances
- Integration effects arising from interactions between heterogeneous subsystems

As such, verification and validation activities shall be conducted incrementally, beginning with subsystem-level testing and progressively advancing toward integrated autonomous flight evaluations. The following sections describe the overall verification philosophy, requirements traceability considerations, and proposed approaches for evaluating software, hardware, communication, and integrated system performance throughout the development lifecycle.

6.1 Verification Philosophy

The verification philosophy adopted for the autonomous drone attitude control system is based upon incremental integration, progressive risk reduction, and subsystem-level validation prior to full system deployment. Because the proposed architecture consists of multiple interacting hardware and software components, verification activities shall be performed in a hierarchical manner in order to minimize integration risk and facilitate identification of failure mechanisms. The overall verification philosophy is guided by the following principles.

6.1.1 Incremental Verification

Subsystems should be verified individually before evaluating integrated system behavior. This approach reduces complexity during debugging activities and permits isolation of individual failure modes. Examples include:

- Verification of learned dynamics models prior to controller integration
- Verification of predictive controller operation prior to communication integration
- Verification of communication interfaces prior to autonomous flight testing

6.1.2 Progressive Integration

System complexity should be increased gradually throughout the development process. Verification activities are therefore expected to proceed through the following stages:

- Software-in-the-loop simulation
- Open-loop controller evaluation
- Closed-loop simulation
- Hardware integration and bench testing
- Hardware-in-the-loop testing
- Tethered flight testing
- Autonomous flight testing

This progression reduces implementation risk and permits assumptions made during simulation studies to be evaluated under increasingly realistic operating conditions.

6.1.3 Risk-Driven Verification

Verification activities should prioritize components and interfaces that present the greatest potential risk to autonomous operation. Particular emphasis should therefore be placed on evaluating:

- Communication latency and synchronization effects
- Computational timing performance
- State estimation accuracy
- Learned model robustness
- Interactions between the embedded computer and flight controller

These areas represent significant sources of uncertainty during transition from simulation to practical implementation. Relevant metrics must be verified to ensure metrics are within safety bounds before flight testing.

6.1.4 Verification of Assumptions

The architecture described throughout this document has been developed under numerous assumptions regarding modeling accuracy, communication behavior, computational performance, and environmental conditions. Verification activities shall therefore seek to determine whether these assumptions remain valid during practical operation. Examples include:

- Validation of controller execution deadlines
- Characterization of communication-induced delays
- Evaluation of estimation performance
- Assessment of robustness to disturbances and environmental effects

6.1.5 Verification Objectives

The primary objectives of verification activities are summarized as follows:

- Demonstrate satisfaction of system requirements
- Identify deficiencies within subsystem interactions
- Quantify implementation-induced uncertainties
- Establish confidence in autonomous operation
- Reduce technical risk prior to flight testing

This philosophy emphasizes systematic risk reduction and recognizes that successful simulation results alone do not guarantee acceptable real-world performance.

6.2 Requirements Traceability

Requirements traceability provides a systematic means of establishing relationships between system requirements, architectural decisions, and verification activities. The purpose of traceability is to ensure that:

- All requirements are addressed by the proposed architecture.
- Appropriate verification methods exist for each requirement.
- Verification activities can be directly related to system objectives.
- Modifications to requirements can be propagated throughout the system development process.

Traceability, therefore, establishes a direct connection between system objectives, architectural implementation, and verification activities.

6.2.1 Traceability Philosophy

The autonomous drone attitude control system follows a top-down requirements flow in which mission objectives are progressively decomposed into functional, performance, interface, and design requirements. These requirements subsequently influence architectural decisions and ultimately

define the verification activities required to demonstrate compliance. The general traceability relationship adopted within this project is illustrated as:

Mission Objective → System Requirement → Architectural Allocation → Verification Method → Verification Evidence

6.2.2 Requirement Allocation

Requirements are allocated to individual subsystems according to their respective responsibilities. Examples include:

- State estimation requirements allocated to the flight controller
- Predictive control requirements allocated to the embedded computer
- Communication requirements allocated to interface services
- Safety requirements allocated to flight controller failsafe mechanisms

6.2.3 Verification Mapping

Each requirement should possess at least one corresponding verification activity. Verification methods may include:

- Analysis
- Inspection
- Demonstration
- Simulation
- Bench testing
- Hardware-in-the-loop testing
- Flight testing

Selection of verification methods depends upon the nature and criticality of the associated requirement.

6.2.4 Traceability Relationships

Requirement	Architectural Allocation	Verification Method
Autonomous attitude stabilization	Predictive controller, flight controller	Closed-loop simulation, flight testing
Real-time controller execution	Embedded Computer	Timing analysis, hardware testing
State estimation availability	Flight controller	Bench testing, integration testing
Communication robustness	Communication interfaces	Latency characterization, fault testing
Safety & failsafe functionality	Flight controller	Demonstration and flight testing

Table 6.1: Range of Update Frequencies for each Subsystem

6.3 Software Verification

Software verification activities are intended to demonstrate that software components have been implemented correctly and perform in accordance with their allocated requirements. Because the proposed autonomous architecture incorporates machine learning models, numerical optimization algorithms, and distributed communication interfaces, software verification must address both functional correctness and real-time execution considerations. Verification activities shall therefore be performed at multiple levels, including unit testing, subsystem verification, simulation-based evaluation, and integrated software testing.

6.3.1 Software Module Verification

Individual software modules shall be verified independently prior to system integration. Examples include:

- Data preprocessing and normalization routines
- Learned dynamics model implementations
- Predictive control algorithms
- Communication interfaces
- Logging and diagnostic utilities

Verification activities may include inspection of software outputs, regression testing, numerical consistency checks, and evaluation under representative operating conditions.

6.3.2 Learned Model Verification

Because the proposed architecture utilizes learned Koopman-based reduced-order models, additional verification activities are required to evaluate model performance and robustness. Representative verification activities include:

- Evaluation of reconstruction accuracy
- Assessment of multi-step prediction performance
- Verification of future latent state time-evolution
- Validation using independent datasets
- Sensitivity analysis under noisy measurements and perturbed initial conditions.

These activities are intended to establish confidence that the learned model adequately captures the system dynamics within the intended operational domain. Particular emphasis should be placed on identifying potential degradation associated with distribution shift and operating conditions that differ significantly from the training data.

6.3.3 Predictive Controller Verification

The model predictive controller shall be verified to ensure proper numerical operation and compliance with design requirements. Representative verification activities include:

- Verification of optimization problem formulation
- Constraint satisfaction evaluation
- Assessment of controller stability characteristics
- Evaluation of reference tracking performance
- Verification of controller behavior under disturbances and modeling errors

Controller verification should additionally include characterization of computational performance, including solver convergence behavior and execution latency.

6.3.4 Software-in-the-Loop Testing

Software-in-the-loop (SIL) simulations shall be used extensively during development to evaluate subsystem interactions in a controlled environment. Representative objectives include:

- Verification of closed-loop functionality
- Identification of software integration issues
- Evaluation of controller robustness
- Assessment of reference tracking performance
- Validation of software interfaces

SIL testing provides an intermediate verification stage prior to hardware integration and significantly reduces implementation risk.

6.3.5 Timing Verification

Because the proposed architecture relies upon real-time execution, software verification shall also include characterization of computational timing performance. Representative measurements include:

- Controller execution time
- Machine learning inference latency
- Communication processing delays
- Scheduling jitter
- End-to-end software latency

These measurements are intended to determine whether software execution remains compatible with required controller update frequencies.

6.3.6 Software Configuration Verification

Software verification should additionally ensure consistency of software configurations and dependencies. Examples include:

- Verification of software version compatibility
- Validation of configuration files and parameter definitions
- Verification of data normalization parameters
- Reproducibility of trained model implementations

This activity is particularly important for learning-based systems in which inconsistencies in preprocessing or parameter configurations may significantly affect autonomous performance. Collectively, these software verification activities are intended to establish confidence that the implemented software architecture satisfies its allocated requirements and is suitable for subsequent integration and flight testing activities.

6.4 Hardware Verification

Hardware verification activities are intended to demonstrate that physical subsystems satisfy their allocated requirements and are capable of supporting autonomous operation under representative operating conditions. Because the proposed architecture consists of multiple heterogeneous hardware components, verification activities shall evaluate individual subsystem performance prior to integrated system testing.

6.4.1 Structural Verification

Structural verification activities are intended to ensure that the airframe and associated mounting hardware satisfy mechanical requirements. Representative activities may include:

- Inspection of manufactured components, mass, and packaging

- Verification of dimensional tolerances
- Evaluation of component mounting integrity
- Assessment of structural rigidity and stiffness
- Evaluation of vibration characteristics and effects

Particular attention should be given to mounting locations of sensors and computational hardware, as excessive vibration may adversely affect state estimation, system identification, and control performance.

6.4.2 Power System Verification

The onboard power system shall be verified to ensure compatibility with computational and propulsion requirements. Representative verification activities include:

- Battery capacity characterization
- Voltage regulation verification
- Measurement of power consumption
- Evaluation of peak current conditions
- Verification of power distribution integrity

These activities are intended to determine whether sufficient electrical power is available for both propulsion and onboard computational hardware during autonomous operation.

6.4.3 Sensing Verification

Sensor verification activities are intended to establish confidence in measurement quality and state estimation inputs. Representative activities include:

- Sensor calibration procedures
- Verification of measurement update rates
- Bias and noise characterization
- Evaluation of sensor repeatability
- Verification of timestamp consistency

Sensor verification may additionally include evaluation under representative vibration and environmental conditions.

6.4.4 Embedded Computing Hardware Verification

The embedded computer shall be verified to ensure that computational resources are sufficient for real-time autonomous operation. Representative verification activities include:

- Measurement of processor utilization
- Memory usage characterization

- Thermal performance evaluation
- Verification of sustained computational workloads
- Identification of potential thermal throttling behavior

These activities are intended to determine whether computational performance remains compatible with controller timing requirements during extended operation. It is worth noting that for real-time testing and deployment, the embedded computer's microSD card, upon which the Linux OS was flashed, was replaced with a 1TB Samsung 980 NVMe SSD; this ensures efficient memory storage and reducing the chances of corrupting data, all of which are potential failure modes during real-time flight testing.

6.4.5 Flight Controller Verification

The flight controller shall be verified to ensure proper operation of low-level flight management functions. Representative activities include:

- Verification of state estimation functionality
- Evaluation of actuator command generation
- Verification of flight modes and failsafe behavior
- Validation of communication interfaces
- Confirmation of timing performance

Because the flight controller retains authority over safety-critical functions, particular emphasis should be placed on verification of emergency procedures and fault handling mechanisms.

6.4.6 Communication Hardware Verification

Communication hardware interfaces shall be verified to ensure reliable information exchange between subsystems. Representative activities include:

- Verification of interface connectivity
- Measurement of communication latency
- Evaluation of message throughput
- Verification of synchronization behavior
- Assessment of communication reliability

These activities are intended to characterize communication performance prior to integrated autonomous testing.

6.4.7 Environmental Hardware Verification

Where practical, hardware verification activities should additionally evaluate subsystem behavior under representative environmental conditions. Examples include:

- Thermal evaluation
- Vibration testing
- Assessment under representative operating loads
- Long-duration operational testing

Such activities provide additional confidence that hardware performance remains acceptable during practical deployment.

6.4.8 Hardware Verification Objectives

The primary objectives of hardware verification activities are summarized as follows:

- Demonstrate proper operation of individual hardware components
- Identify manufacturing or integration deficiencies
- Characterize subsystem limitations
- Quantify hardware-induced uncertainties
- Reduce risk prior to integrated testing and flight operations

Successful completion of hardware verification activities establishes the foundation for subsequent interface verification, integration testing, and autonomous flight evaluations.

Subsystem	**Verification Method**	**Metrics**
Koopman Model	Analysis and SIL	Rollout RMSE, Prediction error
MPC	Simulation	Tracking error, Constraint satisfaction
Jetson	Bench testing	CPU/GPU Utilization, Inference latency
Pixhawk	Bench testing	EKF update rate, Failsafe response
Sensing	Calibration testing	Bias, Noise, Drift
Communication	Integration testing	Latency, Packet loss

Table 6.2: Verification Methods for each Subsystem

6.5 Interface Communication & Integration

The autonomous drone attitude control system utilizes a distributed architecture consisting of multiple hardware and software subsystems executing across heterogeneous computational platforms. Consequently, verification of subsystem interfaces and communication behavior is essential to ensure reliable autonomous operation. The objective of interface verification is to demonstrate that information exchanged between subsystems is accurate, timely, and consistent with architectural assumptions.

6.5.1 Interface Verification Objectives

Interface verification activities are intended to:

- Verify correctness of information exchanged between subsystems
- Confirm compatibility of communication protocols and message formats
- Characterize communication timing behavior
- Identify synchronization issues and integration deficiencies
- Establish confidence in distributed system operation

Because communication performance directly influences estimation and control performance, interface verification shall be considered a critical component of overall system verification.

6.5.2 Communication Interface Validation

Communication interfaces between the flight controller, embedded computer, sensors, and ground station shall be verified prior to autonomous flight operations. Representative verification activities include:

- Verification of physical interface connectivity
- Validation of message formatting and serialization
- Verification of command and telemetry transmission
- Assessment of interface robustness during extended operation
- Evaluation of recovery behavior following communication interruptions

Particular emphasis should be placed on verifying communication pathways associated with safety-critical information and controller commands.

6.5.3 Latency Characterization

The distributed architecture introduces finite communication delays that may influence closed-loop performance. Accordingly, verification activities should characterize:

- State estimation transmission delays;
- Controller command transmission delays;
- End-to-end information latency;
- Message buffering behavior;
- Worst-case communication delays.

These measurements are intended to determine whether communication delays remain compatible with controller timing assumptions.

6.5.4 Timing & Synchronization Verification

Because subsystem execution occurs asynchronously, interface verification shall additionally evaluate synchronization behavior. Representative activities include:

- Verification of timestamp consistency
- Measurement of communication jitter
- Evaluation of clock synchronization behavior
- Identification of stale data conditions
- Assessment of timing variability during representative workloads

Synchronization verification is particularly important for post-flight analysis and for ensuring consistency between state estimates and control actions.

6.5.5 Fault Handling Verification

Communication interfaces shall be evaluated under abnormal operating conditions to ensure graceful degradation and fault tolerance. Representative fault scenarios include:

- Temporary communication interruptions
- Delayed message delivery
- Packet loss conditions
- Embedded computer restarts
- Sensor communication failures

Verification activities should evaluate whether such events can be detected and appropriately managed without resulting in unsafe vehicle behavior.

6.5.6 Interface Verification Success Criteria

Successful interface verification should demonstrate that:

- Communication pathways operate reliably.
- Timing assumptions remain valid.
- Interface protocols are correctly implemented.
- Communication failures are detectable.
- Distributed subsystem interactions remain consistent with architectural expectations.

Completion of interface verification activities establishes confidence that subsystem interactions are sufficiently reliable for subsequent integrated system testing.

6.6 Integration Verification

Integration verification activities are intended to demonstrate that individually verified subsystems operate correctly when combined into the complete autonomous flight architecture. Although individual components may satisfy their respective requirements in isolation, interactions between subsystems frequently introduce emergent behaviors that cannot be fully identified during isolated testing. Consequently, integration verification represents a critical step in reducing implementation risk prior to autonomous flight operations.

6.6.1 Integration Philosophy

The proposed architecture adopts a progressive integration approach in which system complexity is increased incrementally. Subsystems should be integrated in stages, with each stage verifying specific interfaces and functional behaviors before additional components are introduced. Representative integration stages include:

- Learned model and controller integration
- Communication interface integration
- Embedded computer and flight controller integration
- Sensor and state estimation integration
- Hardware-in-the-loop testing
- Flight testing integration

6.6.2 Software & Control Integration

Initial integration activities shall evaluate interactions between software components. Representative objectives include:

- Verification of data flow between software modules
- Validation of state normalization procedures
- Verification of learned model and controller compatibility
- Evaluation of closed-loop software behavior
- Identification of numerical inconsistencies

Because learning-assisted controllers are sensitive to preprocessing assumptions, particular emphasis should be placed on verifying consistency of software configurations and parameter definitions.

6.6.3 Embedded Computer & Flight Controller Integration

Integration activities shall verify interactions between the embedded computer and flight controller. Representative verification activities include:

- Verification of state estimate transmission

- Validation of command interfaces
- Measurement of end-to-end control latency
- Evaluation of controller update frequencies
- Verification of operational mode transitions

6.6.4 Hardware-in-the-Loop Verification

Hardware-in-the-loop (HIL) testing provides an intermediate verification stage between simulation and flight testing. Representative objectives include:

- Evaluation of real hardware timing behavior
- Verification of communication interfaces
- Assessment of computational performance
- Identification of hardware-induced integration effects
- Validation of software execution on target hardware

HIL testing significantly reduces implementation risk by exposing timing and communication effects that are difficult or even impossible in simulation environments.

6.6.5 Safety Integration Verification

Integration verification shall additionally evaluate interactions between autonomy functions and safety mechanisms. Representative activities include:

- Verification of manual override capability
- Evaluation of failsafe activation conditions
- Verification of communication failure handling
- Assessment of controller behavior during abnormal operating conditions
- Validation of emergency operating procedures

Because the flight controller retains authority over safety-critical functions, verification should confirm that autonomy functions cannot inadvertently compromise system safety.

6.6.6 Integration Performance Assessment

Integrated system performance should be evaluated using mission scenarios. Examples include:

- Reference tracking evaluations
- Disturbance rejection assessments
- Extended-duration operation
- Computational workload characterization
- Evaluation under representative environmental conditions

These activities are intended to identify deficiencies arising from subsystem interactions and to

quantify the practical limitations of the implemented architecture.

6.7 Performance Verification

Performance verification activities are intended to determine whether the integrated autonomous drone attitude control system satisfies its allocated functional and performance requirements. Unlike subsystem verification activities, performance verification focuses on the behavior of the complete system under representative operating conditions. The objective of performance verification is to quantify system capabilities, characterize limitations, and establish confidence in the suitability of the proposed architecture for autonomous flight operations.

6.7.1 Performance Verification Objectives

Performance verification activities are intended to demonstrate satisfaction of functional requirements, quantify tracking performance and stability characteristics, characterize computational performance, evaluate robustness to disturbances and uncertainties, and identify practical limitations of the integrated architecture. Verification activities should consider both nominal and off-nominal operating conditions to establish the operational boundaries of the system.

6.7.2 Attitude Tracking Performance

The primary functional objective of the proposed architecture is autonomous attitude stabilization and trajectory tracking. Representative performance metrics include:

- Attitude tracking error
- Root-mean-square tracking error
- Maximum transient error
- Overshoot characteristics
- Settling time
- Steady-state error

Verification activities should evaluate performance across representative reference trajectories, including step inputs, sinusoidal commands, and dynamically varying trajectories.

6.7.3 Disturbance Rejection Performance

Because practical flight environments are subject to external disturbances and modeling uncertainties, performance verification should additionally evaluate robustness characteristics. Representative evaluations include:

- Response to external disturbances
- Sensitivity to modeling errors
- Performance under parameter uncertainty

- Robustness to state estimation errors
- Evaluation under communication delays and timing uncertainties

These activities are intended to establish confidence that acceptable performance can be maintained under realistic operating conditions.

6.7.4 Computational Performance

The distributed architecture imposes computational requirements that directly influence achievable control bandwidth. Performance metrics include:

- Controller execution time
- Machine learning inference latency
- End-to-end control loop latency
- Processor utilization
- Memory utilization
- Timing jitter

These measurements should be compared against allocated timing requirements to determine whether real-time operation can be consistently maintained.

6.7.5 Communication Performance

Communication performance significantly influences distributed autonomous operation. Metrics include:

- Communication latency
- Packet loss rate
- Message throughput
- Timestamp consistency
- Synchronization uncertainty

Verification activities should determine whether communication behavior remains compatible with architectural assumptions and controller requirements.

6.7.6 Reliability & Repeatability

Performance verification should additionally evaluate repeatability across multiple trials and operating conditions. Activities include:

- Repeated simulation studies
- Monte Carlo evaluations
- Sensitivity analyses
- Long-duration operational testing

These activities provide insight into variability in system performance and assist in identifying potential failure modes.

6.7.7 Performance Verification Success Criteria

Performance objectives include:

- Acceptable tolerance for attitude tracking accuracy
- Stable closed-loop operation
- Satisfaction of controller timing requirements
- Reliable communication behavior
- Robustness to representative disturbances

Specific numerical performance thresholds should be derived from system requirements and updated as additional experimental data become available.

6.8 Flight Testing Considerations

Flight testing represents the final stage of system verification and validation and provides the ultimate means of evaluating autonomous system performance under practical operating conditions. Because numerous implementation effects cannot be completely represented through simulation or laboratory testing, flight evaluations are essential for characterizing real-world behavior and identifying limitations of the proposed architecture. However, autonomous flight testing inherently introduces elevated levels of technical and operational risk. Consequently, flight testing activities shall be conducted incrementally and with appropriate safety precautions.

6.8.1 Progressive Flight Testing Philosophy

The proposed architecture adopts a progressive testing methodology in which operational complexity is increased gradually as confidence in subsystem performance improves. Testing phases include:

- Software-in-the-loop testing
- Bench and hardware integration testing
- Hardware-in-the-loop testing
- Tethered flight testing
- Manual flight evaluations
- Assisted autonomy testing
- Fully autonomous flight evaluations

Advancement between testing phases should occur only after satisfactory completion of preceding verification activities.

6.8.2 Test Objectives

Flight testing activities are intended to:

- Validate integrated system behavior;
- Evaluate controller performance under realistic disturbances;
- Characterize communication behavior during flight;
- Assess computational performance under operational workloads;
- Identify deficiencies not observable during simulation.

Particular emphasis should be placed on evaluating assumptions identified throughout previous chapters.

6.8.3 Safety Considerations

Because autonomous flight testing involves experimental control algorithms and distributed computing architectures, safety shall remain the primary consideration throughout all flight activities. Safety measures include:

- Presence of an operator
- Availability of manual override capability
- Verification of flight controller failsafe mechanisms
- Establishment of test termination criteria
- Progressive expansion of the flight envelope

Autonomous functionality should never compromise the independent safety mechanisms provided by the flight controller.

6.8.4 Test Environment Considerations

Initial flight testing activities should be conducted under controlled environmental conditions. Considerations include:

- Moderate wind conditions
- Minimal environmental disturbances
- Adequate operational space
- Controlled access to the testing area
- Availability of recovery and support equipment

Operation under increasingly challenging environmental conditions should only occur following successful evaluation under nominal conditions.

6.8.5 Data Collection & Logging

Comprehensive data collection is essential for post-flight analysis and system improvement. Example data outputs include:

- Vehicle states and state estimates
- Controller outputs
- Communication timestamps
- Computational timing measurements
- Diagnostic and fault information

Sufficient logging capability should be maintained to permit reconstruction of flight events and identification of anomalies.

6.8.6 Success Criteria

Flight test success criteria include:

- Stable autonomous operation
- Acceptable tracking performance
- Satisfaction of timing requirements
- Reliable communication between subsystems
- Proper operation of safety mechanisms

Success criteria should be progressively refined as additional operational experience and experimental data become available.

6.8.7 Flight Test Limitations

Flight testing activities are expected to remain subject to several limitations, including:

- Restricted operating environments
- Limited initial flight envelopes
- Uncertainty associated with environmental disturbances
- Incomplete characterization of rare failure modes

Consequently, flight testing should be viewed as an iterative process in which observations from each testing phase inform future modifications and verification activities.

6.8.8 Future Flight Verification Activities

Future testing activities may include:

- Expanded disturbance evaluations
- Robustness testing under varying environmental conditions

- Long-duration autonomous operation
- Fault injection studies
- Evaluation of perception-based autonomy capabilities

6.9 Future Verification Activities

Although the verification and validation activities described throughout this chapter provide a structured framework for evaluating the proposed autonomous drone architecture, several areas require additional investigation before the system can be considered sufficiently mature for expanded autonomous operation. The current development effort primarily focuses on establishing foundational confidence in the integrated learning-assisted control architecture and identifying major implementation risks associated with distributed autonomous systems. Consequently, future verification activities should seek to progressively expand operational confidence, characterize system limitations, and improve robustness under increasingly realistic operating conditions.

6.9.1 Expanded Simulation Studies

Future verification efforts should include more extensive simulation campaigns to better characterize system performance across a wider range of operating conditions. Activities include:

- Large-scale Monte Carlo simulations
- Evaluation under varying initial conditions
- Assessment under parametric uncertainties
- Robustness studies under environmental disturbances
- Sensitivity analyses involving sensor noise and communication delays

These activities may provide additional insight into the practical operating boundaries of the proposed architecture and assist in identifying potential failure mechanisms prior to flight testing.

6.9.2 Communication & Timing Characterization

Because the proposed architecture relies upon distributed computation, additional verification activities should focus on quantifying communication and timing behavior under representative workloads. Future activities may include:

- End-to-end latency characterization
- Timing jitter measurements
- Packet loss evaluations
- Processor loading studies
- Long-duration communication reliability assessments

These activities are intended to validate assumptions regarding synchronization and real-time execution that cannot be fully evaluated through simulation alone.

6.9.3 Environmental Robustness Testing

Additional verification activities should evaluate system behavior under increasingly realistic environmental conditions. Evaluations may include:

- Wind disturbance testing
- Temperature variation studies
- Vibration sensitivity assessments
- Evaluation under varying lighting and electromagnetic environments
- Long-duration operational testing

Such activities are necessary to determine whether acceptable performance can be maintained outside nominal operating conditions.

6.9.4 Fault Injection & Failure Scenario Testing

Future verification efforts should additionally investigate system behavior during abnormal operating conditions and subsystem failures. Examples include:

- Sensor degradation scenarios
- Communication interruptions
- Delayed or corrupted measurements
- Embedded computer failures
- Controller execution overruns
- Partial subsystem loss

Fault injection studies are particularly important for identifying emergent failure modes and evaluating the effectiveness of existing safety mechanisms and recovery procedures.

6.9.5 Flight Envelope Expansion

Initial flight testing activities are expected to be conducted under relatively conservative operating conditions. Future verification activities should therefore progressively expand the validated flight envelope. Activities may include:

- Increased maneuver aggressiveness
- Higher controller bandwidth evaluations
- Extended mission durations
- Operation under moderate environmental disturbances
- Evaluation of additional autonomous capabilities
- Embedding of autonomy and flight control capabilities onto FPGA arrays

Progressive expansion of the operational envelope provides a systematic means of increasing confidence while maintaining acceptable levels of technical risk.

6.9.6 Learning Model Validation

Because the proposed architecture incorporates learning-assisted dynamics models, future verification activities should continue evaluating model validity and robustness. Studies may include:

- Validation using additional experimental datasets;
- Assessment of performance under distribution shift;
- Evaluation of long-horizon prediction accuracy;
- Investigation of model degradation over time;
- Comparative studies with physics-based models.

These activities are intended to establish confidence in the suitability of learned representations for practical autonomous control applications.

6.9.7 Hardware-in-the-Loop & Real-Time Verification

Additional hardware-in-the-loop studies should be conducted to further reduce implementation risk prior to extensive flight testing. Activities include:

- Extended-duration hardware evaluations
- Real-time computational stress testing
- Verification of timing assumptions under representative workloads
- Evaluation of communication behavior under operational conditions
- Integration testing involving additional onboard hardware

These activities provide an intermediate verification stage between simulation and practical deployment and assist in identifying implementation issues that may not be apparent during purely software-based studies.

6.9.8 Requirements & Traceability Maintenance

As the architecture evolves and additional capabilities are incorporated, future verification activities should maintain consistency between requirements, implementation, and verification evidence. Future efforts should, therefore, include:

- Periodic review of system requirements
- Updates to verification matrices and traceability documentation
- Re-verification following significant architectural modifications
- Configuration management of software and hardware baselines

Maintaining requirements traceability is essential for preserving consistency throughout future system development activities.

6.9.9 Long-Term Verification Objectives

Long-term verification activities should ultimately seek to establish confidence in the practical applicability of learning-assisted autonomous flight control systems. Objectives include:

- Demonstration of reliable integrated autonomous operation
- Validation under representative environmental conditions
- Characterization of operational limitations
- Evaluation of robustness and repeatability
- Identification of pathways toward higher levels of autonomy

Accordingly, future verification activities will play a critical role in reducing technical uncertainty, improving system robustness, and guiding the continued maturation of the proposed autonomous drone architecture.